

6. 동기화 - synchronized

#1.인강/0.자바/5.자바-고급1편

- /출금 예제 - 시작
- /동시성 문제
- /임계 영역
- /synchronized 메서드
- /synchronized 코드 블록
- /문제와 풀이
- /정리

출금 예제 - 시작

멀티스레드를 사용할 때 가장 주의해야 할 점은, 같은 자원(리소스)에 여러 스레드가 동시에 접근할 때 발생하는 동시성 문제이다. 참고로 여러 스레드가 접근하는 자원을 **공유 자원**이라 한다. 대표적인 공유 자원은 인스턴스의 필드(멤버 변수)이다.

멀티스레드를 사용할 때는 이런 공유 자원에 대한 접근을 적절하게 동기화(synchronization)해서 동시성 문제가 발생하지 않게 방지하는 것이 중요하다.

동시성 문제가 어떤 문제인지 이해하기 위해 간단한 은행 출금 예제를 하나 만들어보자.

```
package thread.sync;

public interface BankAccount {

    boolean withdraw(int amount);

    int getBalance();
}
```

- `BankAccount` 인터페이스이다. 앞으로 이 인터페이스의 구현체를 점진적으로 발전시키면서 문제를 해결할 예정이다.
- `withdraw(amount)`: 계좌의 돈을 출금한다. 출금할 금액을 매개변수로 받는다.
 - 계좌의 잔액이 출금할 금액보다 많다면 출금에 성공하고, `true`를 반환한다.
 - 계좌의 잔액이 출금할 금액보다 적다면 출금에 실패하고, `false`를 반환한다.
- `getBalance()`: 계좌의 잔액을 반환한다.

```

package thread.sync;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class BankAccountV1 implements BankAccount {
    private int balance;
    //volatile private int balance;

    public BankAccountV1(int initialBalance) {
        this.balance = initialBalance;
    }

    @Override
    public boolean withdraw(int amount) {
        log("거래 시작: " + getClass().getSimpleName());

        log("[검증 시작] 출금액: " + amount + ", 잔액: " + balance);
        if (balance < amount) {
            log("[검증 실패] 출금액: " + amount + ", 잔액: " + balance);
            return false;
        }
        log("[검증 완료] 출금액: " + amount + ", 잔액: " + balance);
        sleep(1000); // 출금에 걸리는 시간으로 가정
        balance = balance - amount;
        log("[출금 완료] 출금액: " + amount + ", 변경 잔액: " + balance);

        log("거래 종료");
        return true;
    }

    @Override
    public int getBalance() {
        return balance;
    }
}

```

- BankAccountV1 은 BankAccount 인터페이스를 구현한다.
- 생성자를 통해 계좌의 초기 잔액을 저장한다.
- int balance: 계좌의 잔액 필드
- withdraw(amount): 검증과 출금 2가지 단계로 나누어진다.

- **검증 단계:** 출금액과 잔액을 비교한다. 만약 출금액이 잔액보다 많다면 문제가 있으므로 검증에 실패하고, `false`를 반환한다.
- **출금 단계:** 검증에 통과하면 잔액이 출금액보다 많으므로 출금할 수 있다. 잔액에서 출금액을 빼고 출금을 완료하면, 성공이라는 의미의 `true`를 반환한다.
- `getBalance()`: 잔액을 반환한다.

```
package thread.sync;

public class WithdrawTask implements Runnable {

    private BankAccount account;
    private int amount;

    public WithdrawTask(BankAccount account, int amount) {
        this.account = account;
        this.amount = amount;
    }

    @Override
    public void run() {
        account.withdraw(amount);
    }

}
```

- 출금을 담당하는 `Runnable` 구현체이다. 생성시 출금할 계좌(`account`)와 출금할 금액(`amount`)을 저장해 둔다.
- `run()`을 통해 스레드가 출금을 실행한다.

```
package thread.sync;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class BankMain {

    public static void main(String[] args) throws InterruptedException {
        BankAccount account = new BankAccountV1(1000);

        Thread t1 = new Thread(new WithdrawTask(account, 800), "t1");
```

```

Thread t2 = new Thread(new WithdrawTask(account, 800), "t2");
t1.start();
t2.start();

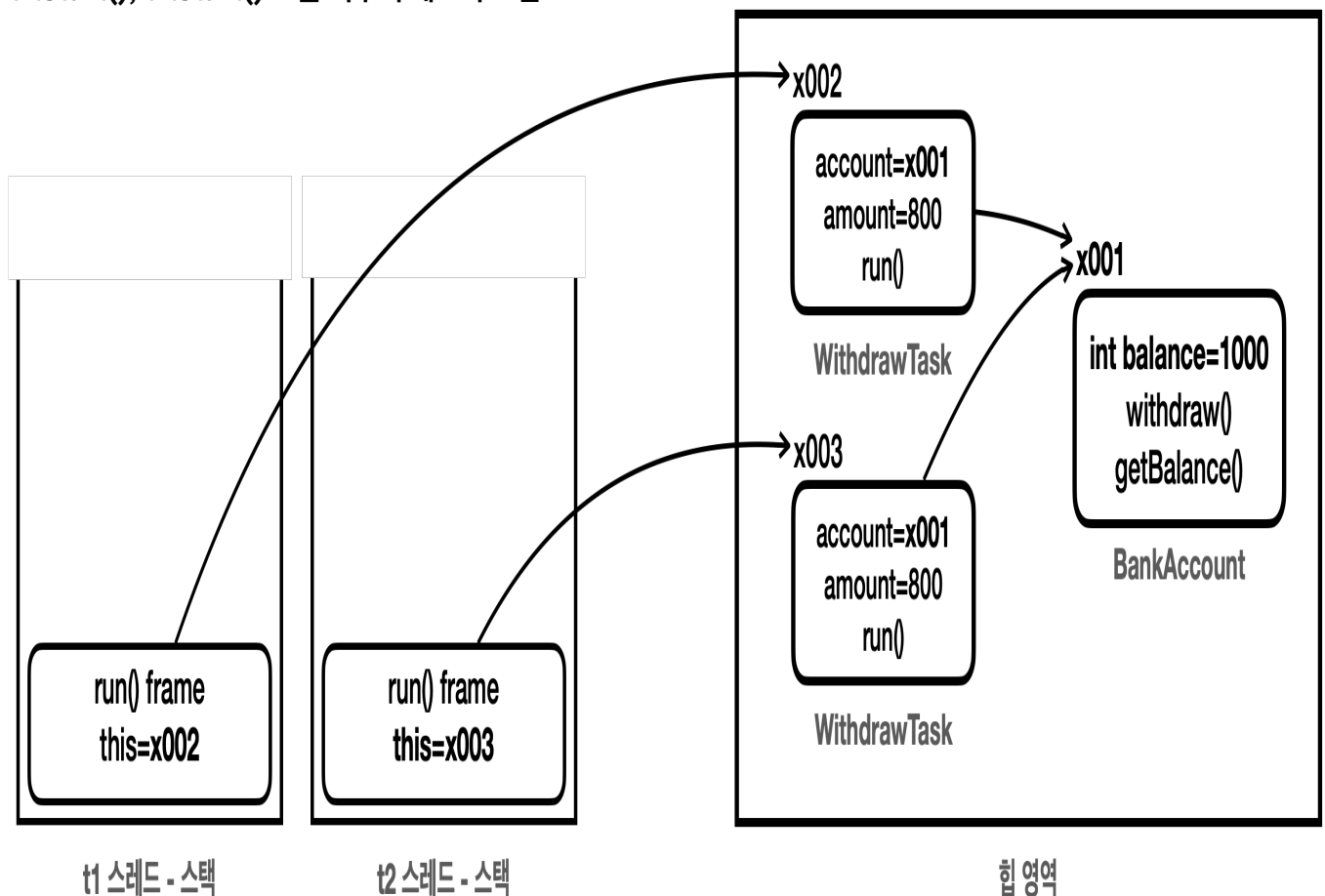
sleep(500); // 검증 완료까지 잠시 대기
log("t1 state: " + t1.getState());
log("t2 state: " + t2.getState());

t1.join();
t2.join();
log("최종 잔액: " + account.getBalance());
}
}

```

- new BankAccountV1(1000) 을 통해 초기 잔액을 1000 원으로 설정한다.
- main 스레드는 t1, t2 스레드를 만든다. 만든 스레드들은 같은 계좌에 각각 800 원의 출금을 시도한다.
- main 스레드는 join() 을 사용해서 t1, t2 스레드가 출금을 완료한 이후에 최종 잔액을 확인한다.

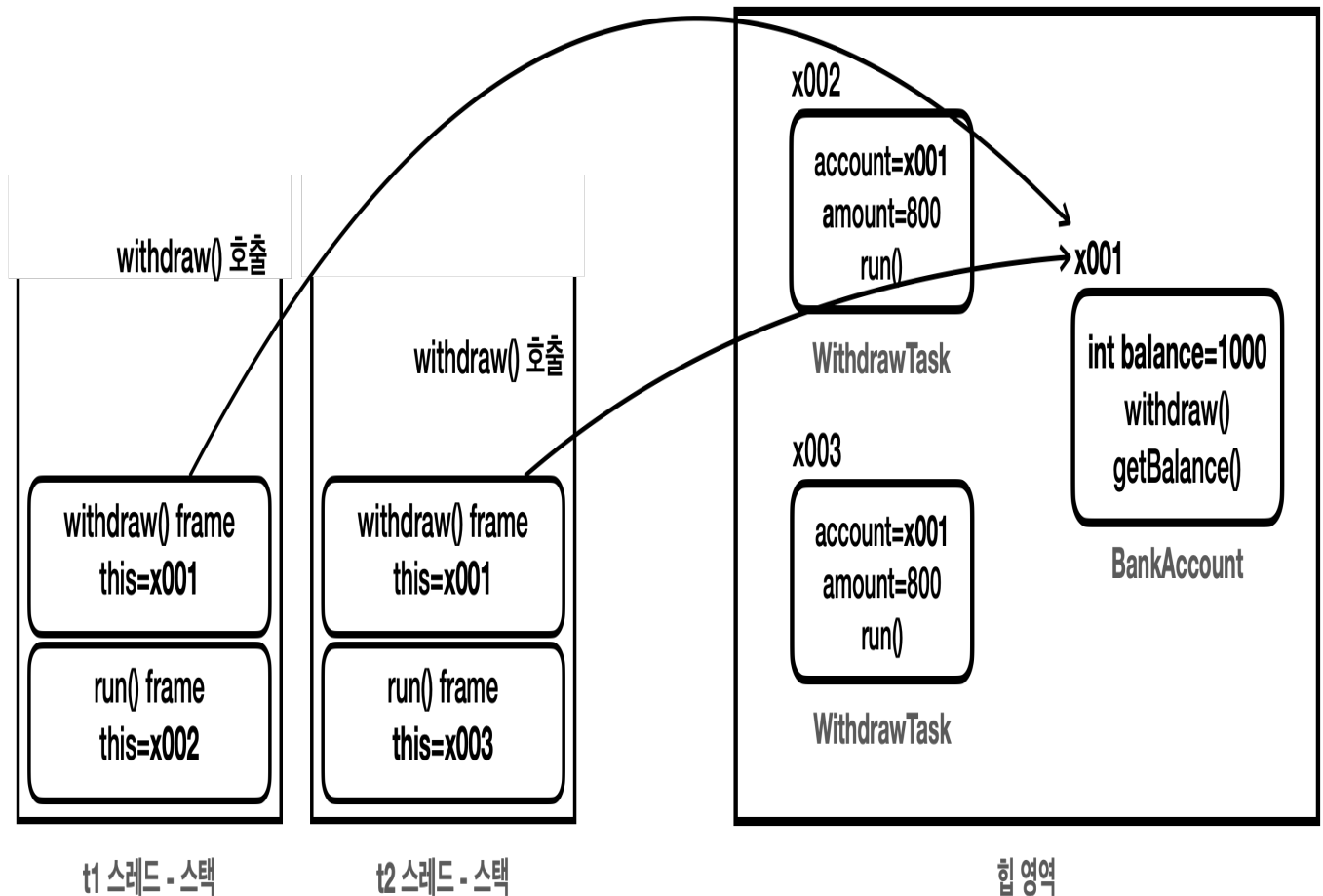
t1.start(), t2.start() 호출 직후의 메모리 그림



- 각각의 스레드의 스택에서 `run()` 이 실행된다.

- t1 스레드는 WithdrawTask(x002) 인스턴스의 run() 을 호출한다.
- t2 스레드는 WithdrawTask(x003) 인스턴스의 run() 을 호출한다.
- 스택 프레임의 this 에는 호출한 메서드의 인스턴스 참조가 들어있다.
- 두 스레드는 같은 계좌(x001)에 대해서 출금을 시도한다.

참고: 그림에서는 편의상 BankAccountV1 대신에 BankAccount 라고 표현하겠다.



- t1 스레드의 run() 에서 withdraw() 를 실행한다.
- 거의 동시에 t2 스레드의 run() 에서 withdraw() 를 실행한다.
- t1 스레드와 t2 스레드는 같은 BankAccount(x001) 인스턴스의 withdraw() 메서드를 호출한다.
- 따라서 두 스레드는 같은 BankAccount(x001) 인스턴스에 접근하고 또 x001 인스턴스에 있는 잔액(balance) 필드도 함께 사용한다.

실행 결과

```

11:09:40.185 [      t1] 거래 시작: BankAccountV1
11:09:40.185 [      t2] 거래 시작: BankAccountV1
11:09:40.192 [      t1] [검증 시작] 출금액: 800, 잔액: 1000
11:09:40.192 [      t2] [검증 시작] 출금액: 800, 잔액: 1000
11:09:40.192 [      t1] [검증 완료] 출금액: 800, 잔액: 1000
  
```

```

11:09:40.192 [      t2] [검증 완료] 출금액: 800, 잔액: 1000
11:09:40.673 [    main] t1 state: TIMED_WAITING
11:09:40.673 [    main] t2 state: TIMED_WAITING
11:09:41.195 [      t1] [출금 완료] 출금액: 800, 변경 잔액: 200
11:09:41.195 [      t1] 거래 종료
11:09:41.197 [      t2] [출금 완료] 출금액: 800, 변경 잔액: -600
11:09:41.197 [      t2] 거래 종료
11:09:41.200 [    main] 최종 잔액: -600

```

- **참고:** 여기서는 t1 스레드가 먼저 실행되었다. 그런데 실행 환경에 따라서 t1, t2가 완전히 동시에 실행될 수도 있다. 이 경우 출금액은 같고, 잔액은 200원이 된다. 이 부분은 바로 뒤에서 설명한다.

동시성 문제

이 시나리오는 악의적인 사용자가 2대의 PC에서 동시에 같은 계좌의 돈을 출금한다고 가정한다.

- t1, t2, 스레드는 거의 동시에 실행되지만, 아주 약간의 차이로 t1 스레드가 먼저 실행되고, t2 스레드가 그 다음에 실행된다고 가정하겠다.
- 처음 계좌의 잔액은 1000원이다. t1 스레드가 800원을 출금하면 잔액은 200원이 남는다.
- 이제 계좌의 잔액은 200원이다. t2 스레드가 800원을 출금하면 잔액보다 더 많은 돈을 출금하게 되므로 출금에 실패해야 한다.

그런데 실행 결과를 보면 기대와는 다르게 t1, t2는 각각 800원씩 총 1600원 출금에 성공한다.

계좌의 잔액은 -600원이 되어있고, 계좌는 예상치 못하게 마이너스 금액이 되어버렸다.

악의적인 사용자는 2대의 PC를 통해 자신의 계좌에 있는 1000원 보다 더 많은 금액인 1600원 출금에 성공한다.

분명히 계좌를 출금할 때 잔고를 체크하는 로직이 있는데도 불구하고, 왜 이런 문제가 발생했을까?

계좌 출금시 잔고 체크 로직

```

if (balance < amount) {
    log("[검증 실패] 출금액: " + amount + ", 잔액: " + balance);
    return false;
}

```

참고: balance 값에 volatile 을 도입하면 문제가 해결되지 않을까? 그렇지 않다. volatile 은 한 스레드가 값을 변경했을 때 다른 스레드에서 변경된 값을 즉시 볼 수 있게 하는 메모리 가시성의 문제를 해결할 뿐이다. 예를 들어 t1 스레드가 balance 의 값을 변경했을 때, t2 스레드에서 balance 의 변경된 값을 즉시 확인해도 여전히 같은 문제가 발생한다. 이 문제는 메모리 가시성 문제를 해결해도 여전히 발생한다.

동시성 문제

왜 이런 문제가 발생하는지 하나씩 천천히 분석해보자.

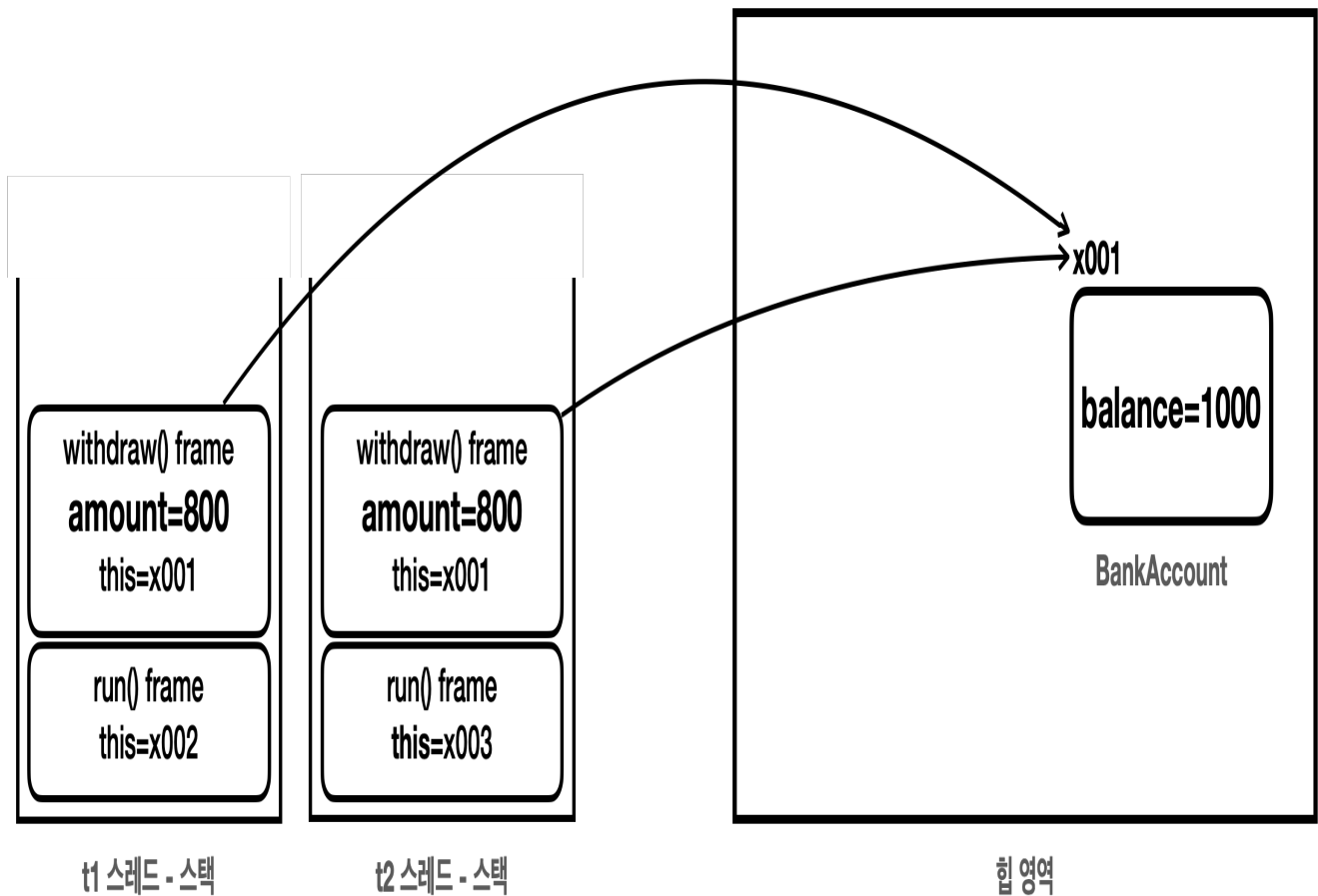
t1, t2 순서로 실행 가정

t1 이 아주 약간 빠르게 실행되는 경우를 먼저 알아보자. 설명을 단순화 하기 위해 일부 그림은 생략했다.

t1, t2 순서로 실행 가정

```
withdraw { t2  
  if (balance < amount) { t1  
    return false;  
  }  
  sleep(1000);  
  balance = balance - amount;  
}
```

t1: 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.



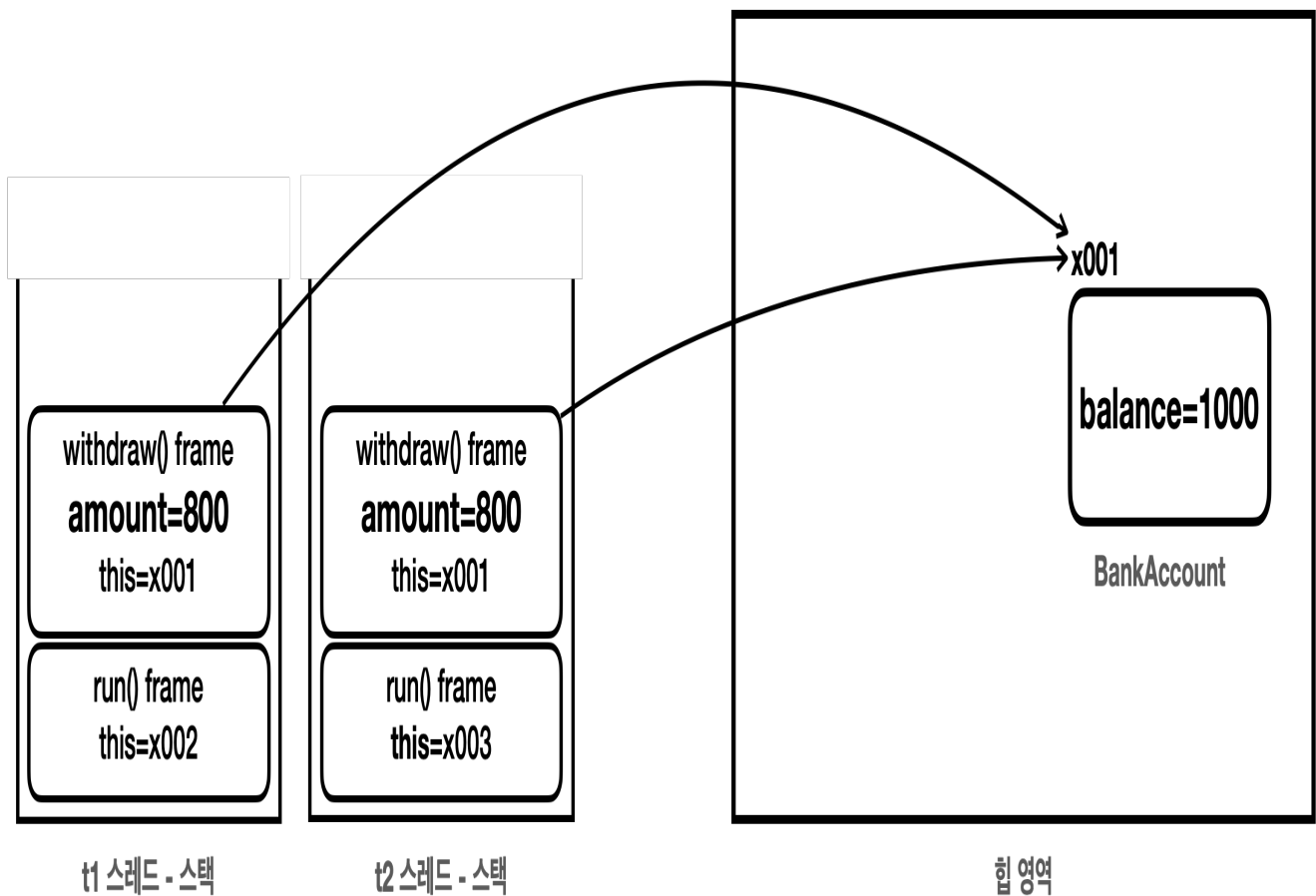
- t1 이 약간 먼저 실행되면서, 출금을 시도한다.
- t1 이 출금 코드에 있는 검증 로직을 실행한다. 이때 잔액이 출금 액수보다 많은지 확인한다.
 - 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.

```

withdraw {
  if (balance < amount) { t2
    return false;
  }
  sleep(1000); t1
  balance = balance - amount;
}

```

t2: 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.



- t1 : 출금 검증 로직을 통과해서 출금을 위해 잠시 대기중이다. 출금에 걸리는 시간으로 생각하자.
- t2 : 검증 로직을 실행한다. 잔액이 출금 금액보다 많은지 확인한다.
 - 잔액[1000]이 출금액[800] 보다 많으므로 통과한다.

바로 이 부분이 문제다! t1이 아직 잔액(balance)를 줄이지 못했기 때문에 t2는 검증 로직에서 현재 잔액을 1000원으로 확인한다.

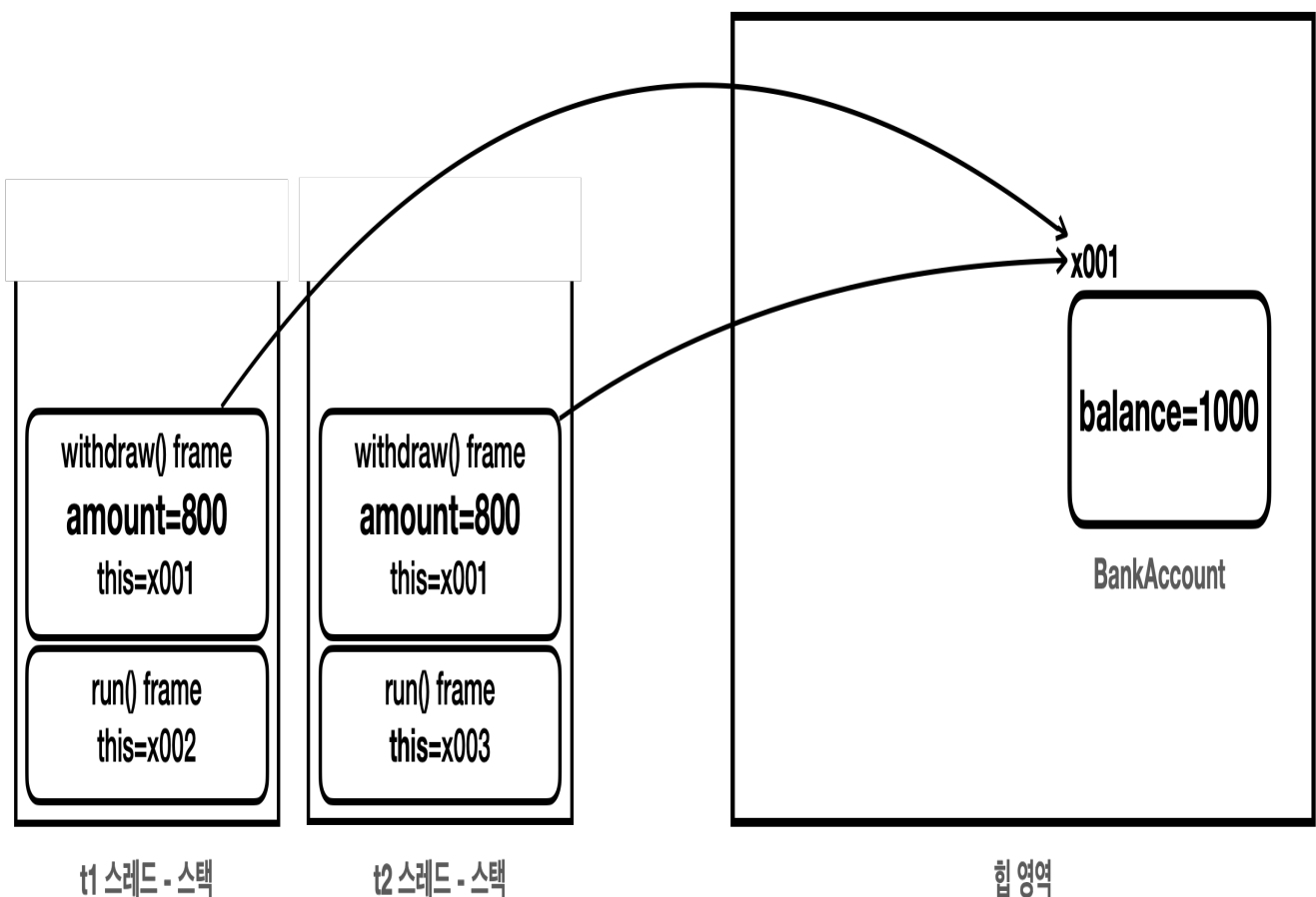
t1 이 검증 로직을 통과하고 바로 잔액을 줄였다면 이런 문제가 발생하지 않겠지만, t1 이 검증 로직을 통과하고 잔액을 줄이기도 전에 먼저 t2 가 검증 로직을 확인한 것이다.

그렇다면 `sleep(1000)` 코드를 빼면 되지 않을까? 이렇게하면 `t1` 이 검증 로직을 통과하고 바로 잔액을 줄일 수 있을 것 같다. 하지만 `t1` 이 검증 로직을 통과하고 `balance = balance - amount` 를 계산하기 직전에 `t2` 가 실행 되면서 검증 로직을 통과할 수도 있다. `sleep(1000)` 은 단지 이런 문제를 쉽게 확인하기 위해 넣었을 뿐이다.

```
withdraw {  
  if (balance < amount) {  
    return false;  
  }  
  sleep(1000); t1 t2  
  balance = balance - amount;  
}
```

t1: 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.

t2: 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.



- 결과적으로 `t1`, `t2` 모두 검증 로직을 통과하고, 출금을 위해 잠시 대기중이다. 출금에 걸리는 시간으로 생각하자.

```

withdraw {
  if (balance < amount) {
    return false;
  }
  sleep(1000);
  balance = balance - amount;
}

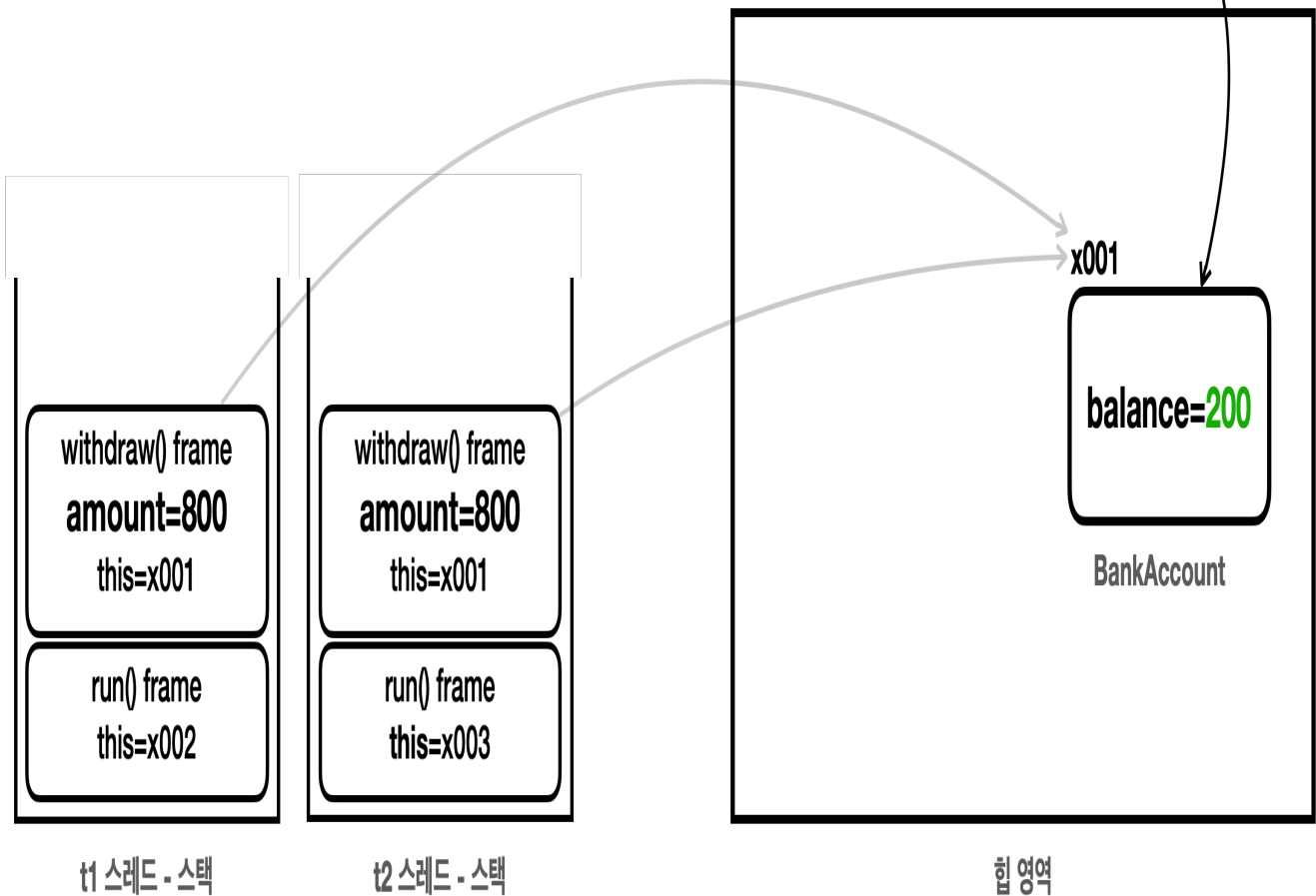
```

t2

t1

t1: balance[200] = balance[1000] - amount[800]

200



t1 스레드 - 스택

t2 스레드 - 스택

힙 영역

- t1 은 800원을 출금하면서, 잔액을 1000원에서 출금 액수인 800원 만큼 차감한다. 이제 계좌의 잔액은 200원이 된다.

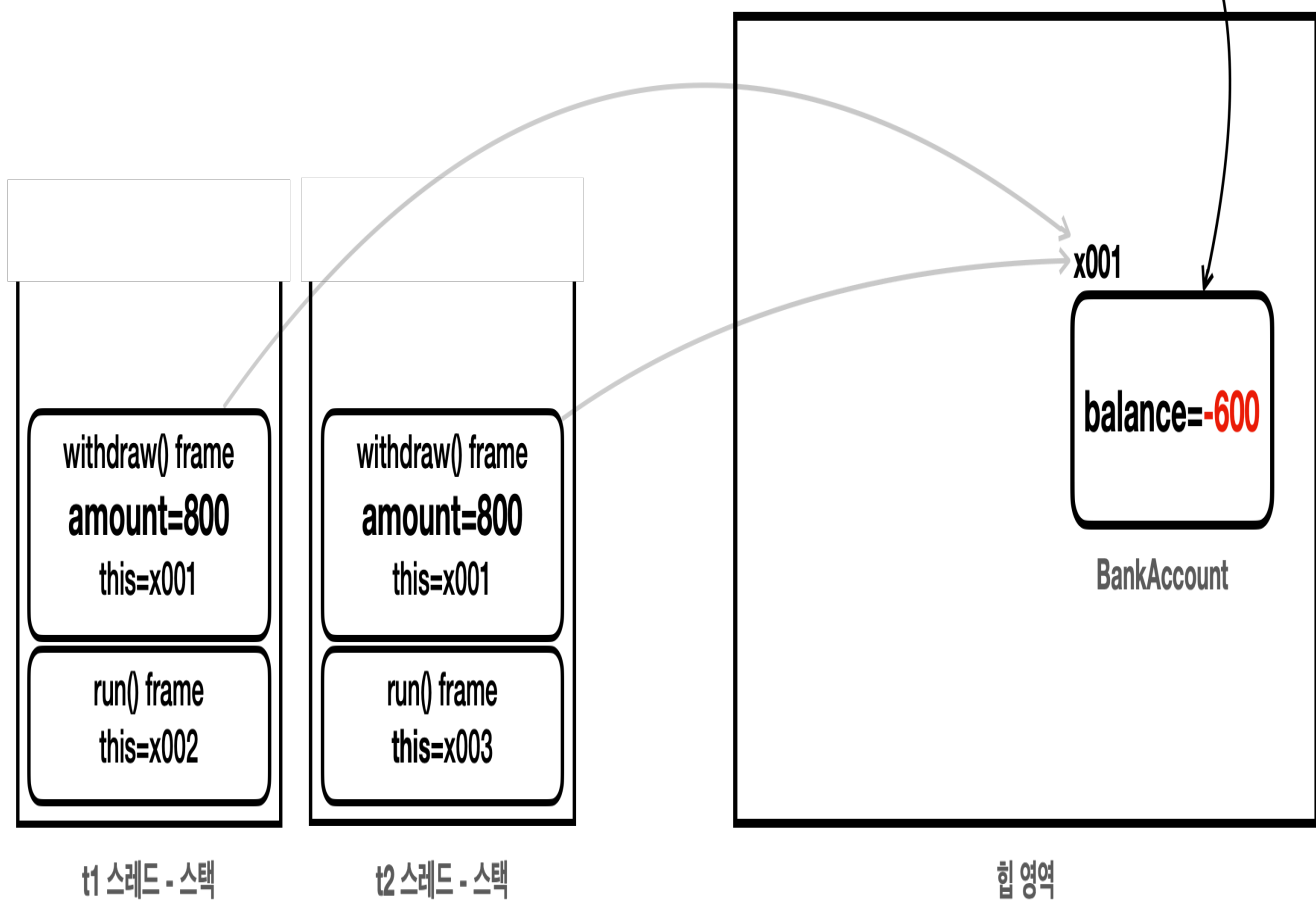
```

withdraw {
  if (balance < amount) {
    return false;
  }
  sleep(1000);
  balance = balance - amount;
}

```

t2: $\text{balance}[-600] = \text{balance}[200] - \text{amount}[800]$

-600



- t2는 800원을 출금하면서, 잔액을 200원에서 출금 액수인 800원 만큼 차감한다. 이제 잔액은 -600원이 된다.

```

withdraw {
  if (balance < amount) {
    return false;
  }
  sleep(1000);
  balance = balance - amount;
}

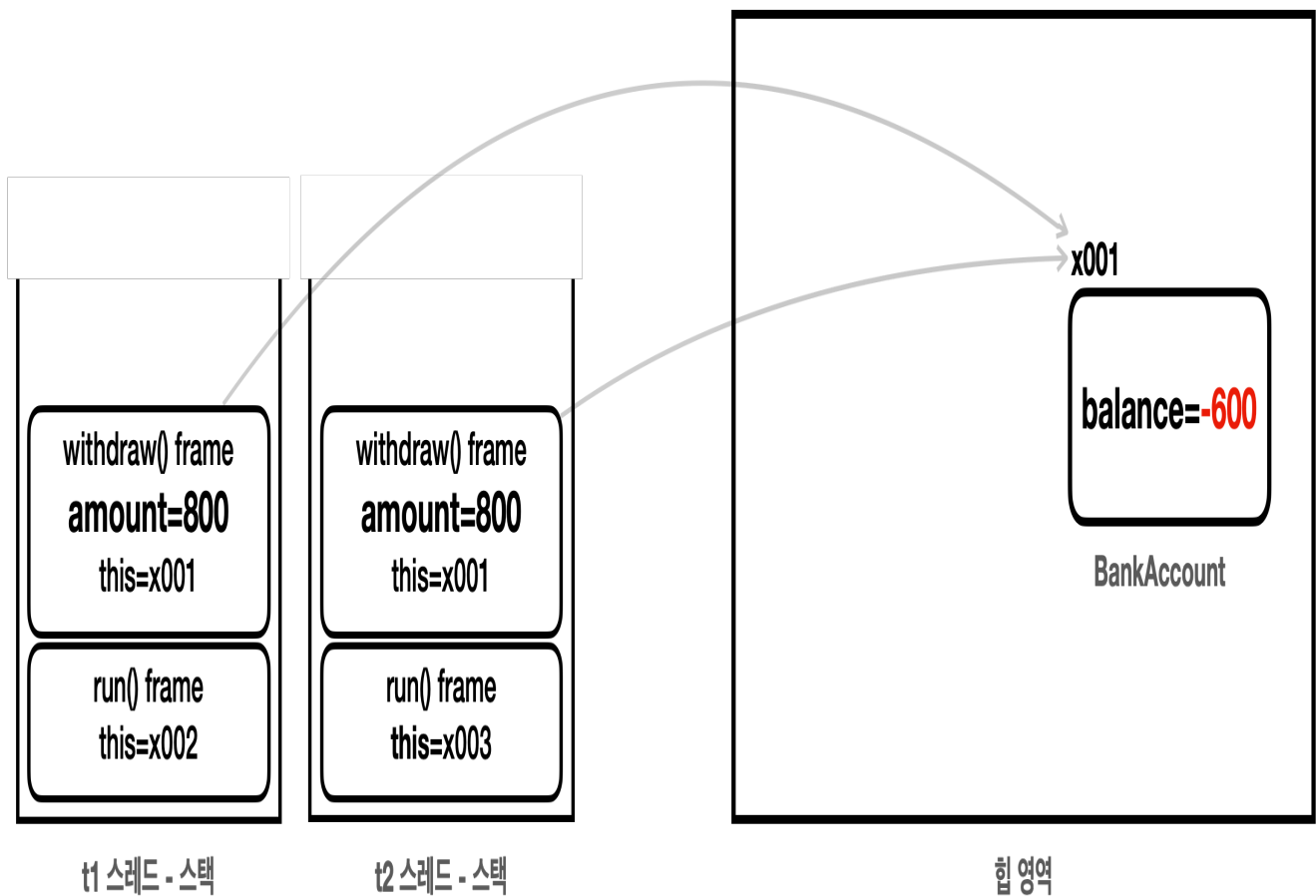
```

t1 t2

t1: 800원 출금 완료

t2: 800원 출금 완료

원금 1000원, 최종 잔액 -600원



결과

- t1: 800원 출금 완료
- t2: 800원 출금 완료
- 처음 원금은 1000원이었는데, 최종 잔액은 -600원이 된다.
- 은행 입장에서 마이너스 잔액이 있으면 안된다!

실행 결과

11:09:40.185 [

t2] 거래 시작: BankAccountV1

```
11:09:40.185 [      t1] 거래 시작: BankAccountV1
11:09:40.192 [      t1] [검증 시작] 출금액: 800, 잔액: 1000
11:09:40.192 [      t2] [검증 시작] 출금액: 800, 잔액: 1000
11:09:40.192 [      t1] [검증 완료] 출금액: 800, 잔액: 1000
11:09:40.192 [      t2] [검증 완료] 출금액: 800, 잔액: 1000
11:09:40.673 [    main] t1 state: TIMED_WAITING
11:09:40.673 [    main] t2 state: TIMED_WAITING
11:09:41.195 [      t1] [출금 완료] 출금액: 800, 변경 잔액: 200
11:09:41.195 [      t1] 거래 종료
11:09:41.197 [      t2] [출금 완료] 출금액: 800, 변경 잔액: -600
11:09:41.197 [      t2] 거래 종료
11:09:41.200 [    main] 최종 잔액: -600
```

이것은 우리가 기대한 결과가 아니다.

t1, t2 동시에 실행 가정

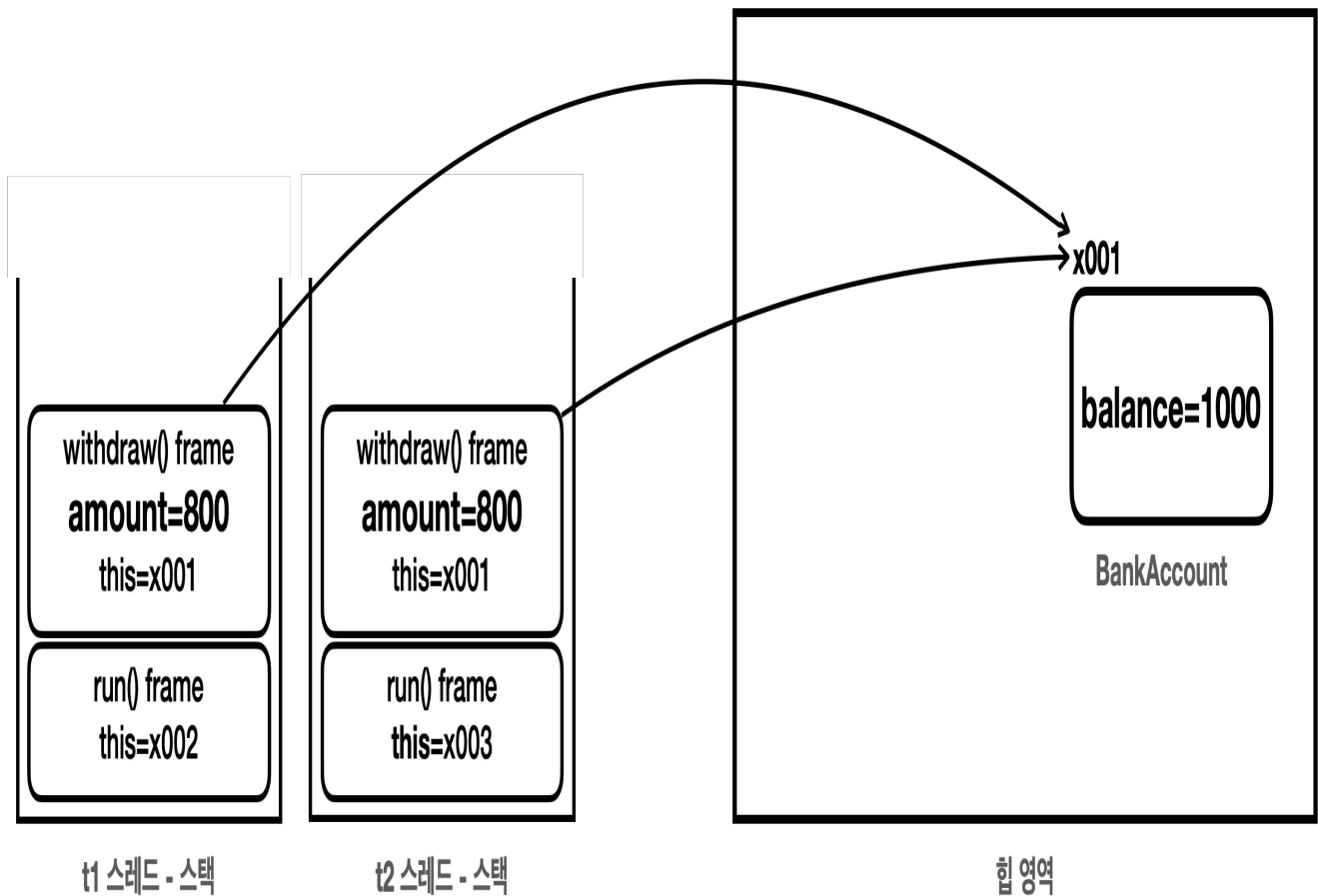
t1, t2가 완전히 동시에 실행되는 상황을 알아보자.

t1, t2 동시에 실행 가정

```
withdraw {  
  if (balance < amount) { t1 t2  
    return false;  
  }  
  sleep(1000);  
  balance = balance - amount;  
}
```

t1: 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.

t2: 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.



- t1, t2 는 동시에 검증 로직을 실행한다. 잔액이 출금 금액보다 많은지 확인한다.
 - 잔액[1000]이 출금액[800] 보다 많으므로 둘다 통과한다.

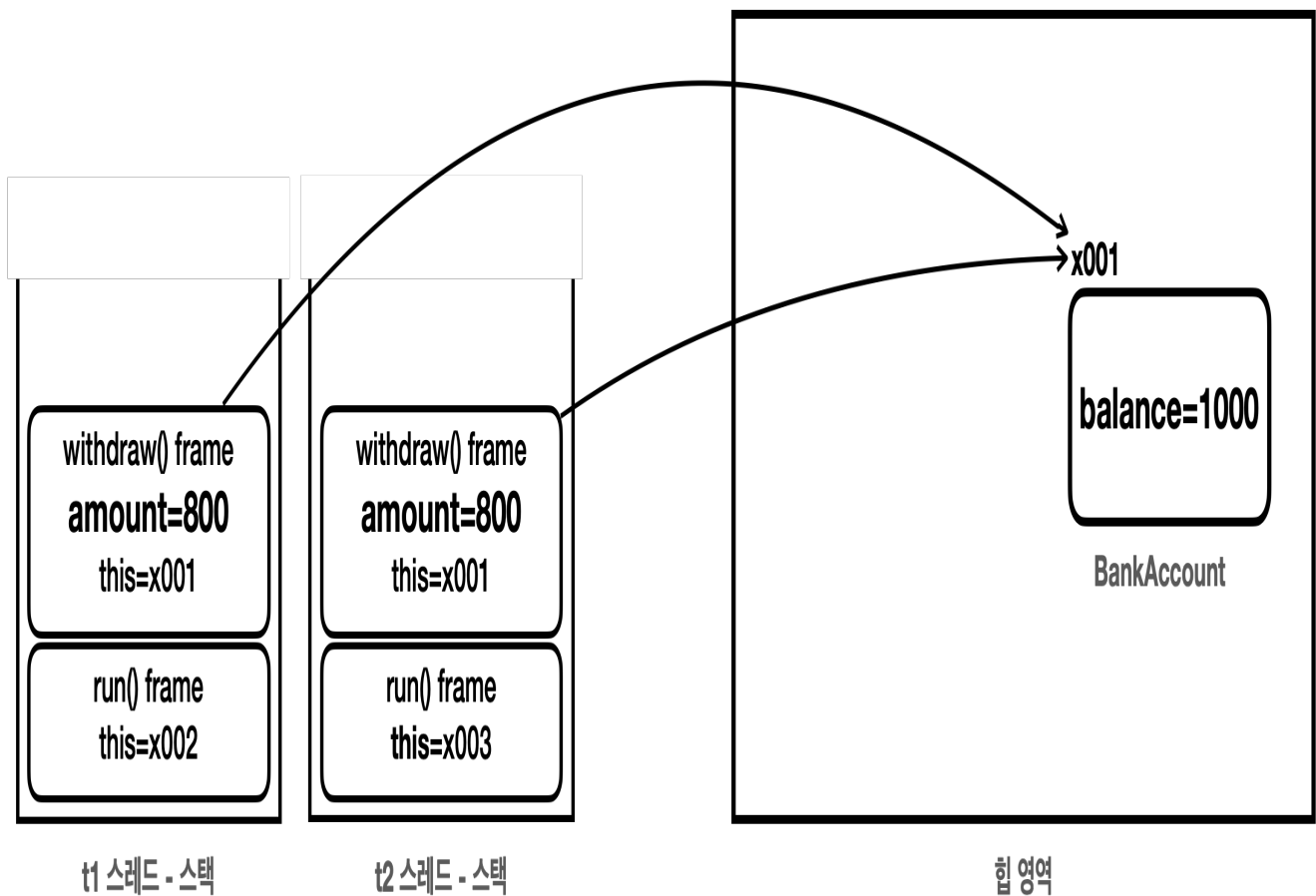
```

withdraw {
  if (balance < amount) {
    return false;
  }
  sleep(1000); t1 t2
  balance = balance - amount;
}

```

t1: 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.

t2: 잔액[1000]이 출금액[800] 보다 많으므로 검증 로직을 통과한다.



- 결과적으로 t1, t2 모두 검증 로직을 통과하고, 출금을 위해 잠시 대기중이다. 출금에 걸리는 시간으로 생각하자.

```

withdraw {
  if (balance < amount) {
    return false;
  }
  sleep(1000);
  balance = balance - amount;
}

```

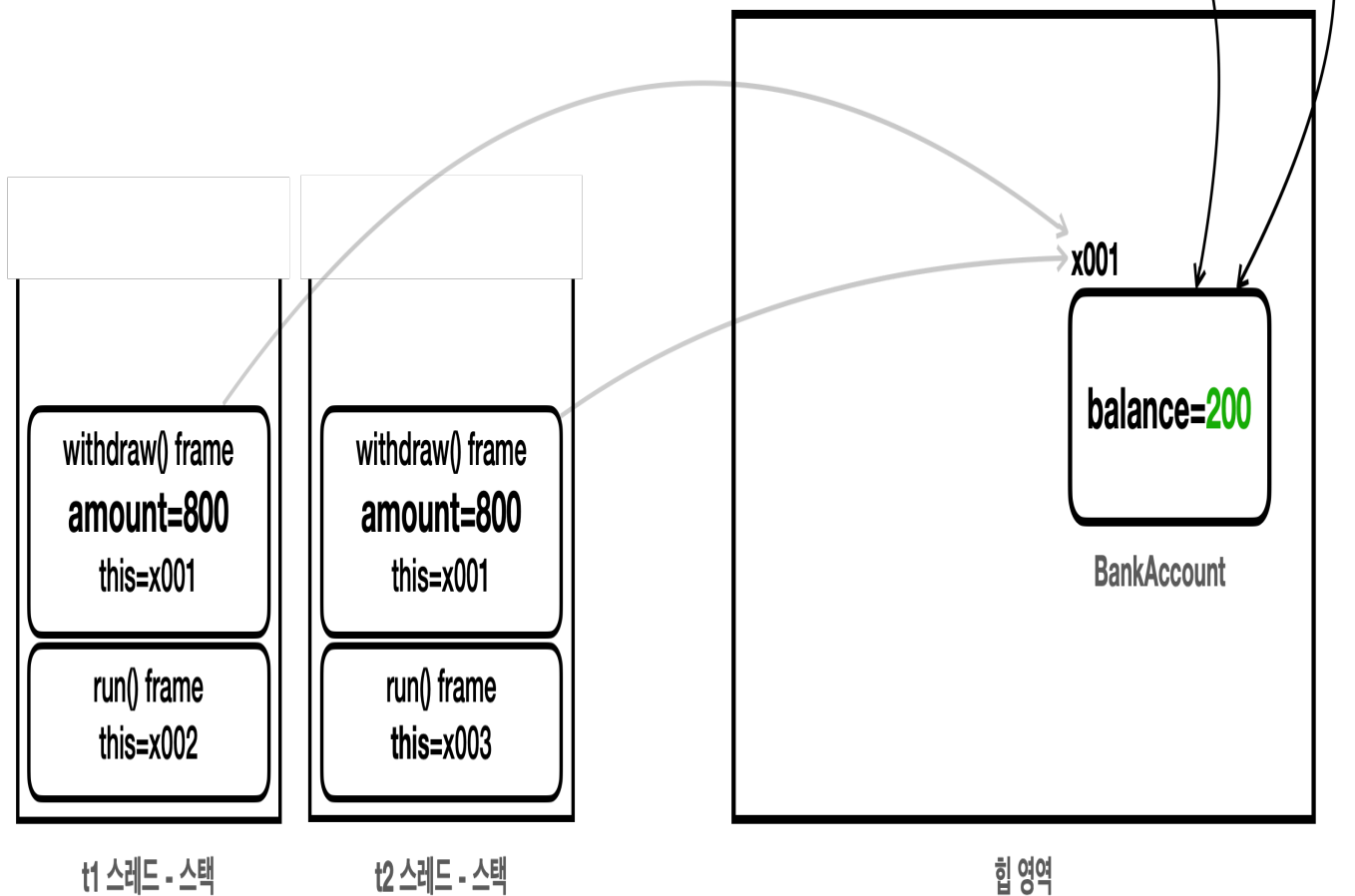
t1 t2

t1: balance[200] = balance[1000] - amount[800]

t2: balance[200] = balance[1000] - amount[800]

200

200



- t1 은 800원을 출금하면서, 잔액을 1000원에서 출금 액수인 800원 만큼 차감한다. 이제 잔액은 200원이 된다.
- t2 은 800원을 출금하면서, 잔액을 1000원에서 출금 액수인 800원 만큼 차감한다. 이제 잔액은 200원이 된다.
- t1, t2 가 동시에 실행되기 때문에 둘다 잔액 (balance)을 확인하는 시점에 잔액은 1000원이다!
- t1, t2 둘다 동시에 계산된 결과를 잔액에 반영하는데, 둘다 계산 결과인 200원을 반영하므로 최종 잔액은 200원이 된다.

```
balance = balance - amount;
```

이 코드는 다음의 단계로 이루어진다.

1. 계산을 위해 오른쪽에 있는 `balance` 값과 `amount` 값을 조회한다.
 2. 두 값을 계산한다.
 3. 계산 결과를 왼쪽의 `balance` 변수에 저장한다.
- 여기서 1번 단계의 `balance` 값을 조회할 때 `t1`, `t2` 두 스레드가 동시에 `x001.balance`의 필드 값을 읽는다. 이때 값은 `1000`이다. 따라서 두 스레드는 모두 잔액을 1000원으로 인식한다.
 - 2번 단계에서 두 스레드 모두 `1000 - 800`을 계산해서 `200`이라는 결과를 만든다.
 - 3번 단계에서 두 스레드 모두 `balance = 200`을 대입한다.

```

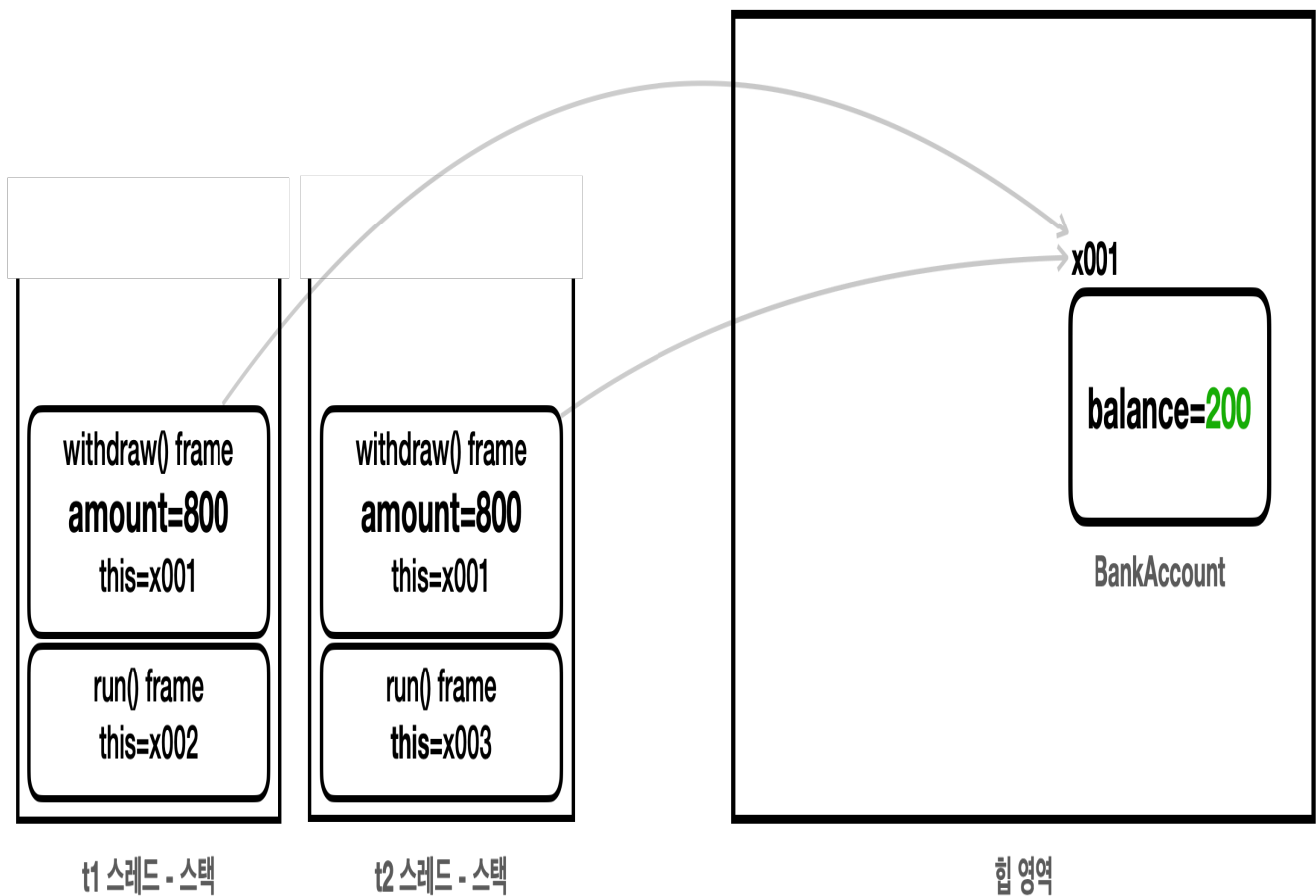
withdraw {
  if (balance < amount) {
    return false;
  }
  sleep(1000);
  balance = balance - amount;
}

```

t1: 800원 출금 완료

t2: 800원 출금 완료

원금 1000원, 최종 잔액 200원



결과

- t1 : 800원 출금완료
- t2 : 800원 출금완료
- 원래 원금이 1000원이었는데, 최종 잔액은 200원이 된다.
- 은행 입장에서 보면 총 1600원이 빠져나갔는데, 잔액은 800원만 줄어들었다. 800원이 감쪽같이 어디론가 사라진 것이다!

실행 결과

```

12:31:10.885 [      t2] 거래 시작: BankAccountV1
12:31:10.885 [      t1] 거래 시작: BankAccountV1
12:31:10.892 [      t1] [검증 시작] 출금액: 800, 잔액: 1000
12:31:10.892 [      t2] [검증 시작] 출금액: 800, 잔액: 1000
12:31:10.893 [      t1] [검증 완료] 출금액: 800, 잔액: 1000
12:31:10.893 [      t2] [검증 완료] 출금액: 800, 잔액: 1000
12:31:11.370 [    main] t1 state: TIMED_WAITING
12:31:11.370 [    main] t2 state: TIMED_WAITING
12:31:11.898 [      t2] [출금 완료] 출금액: 800, 변경 잔액: 200
12:31:11.898 [      t1] [출금 완료] 출금액: 800, 변경 잔액: 200
12:31:11.899 [      t2] 거래 종료
12:31:11.899 [      t1] 거래 종료
12:31:11.905 [    main] 최종 잔액: 200

```

실행 결과에서 시간이 완전히 같다는 사실을 통해 두 스레드가 같이 실행된 것을 대략 확인할 수 있다.

이것은 우리가 기대한 결과가 아니다.

이 문제가 왜 발생했고, 또 이런 문제를 어떻게 해결할 수 있을까?

임계 영역

이런 문제가 발생한 근본 원인은 여러 스레드가 함께 사용하는 공유 자원을 여러 단계로 나누어 사용하기 때문이다.

- **1. 검증 단계:** 잔액(balance)이 출금액(amount) 보다 많은지 확인한다.
- **2. 출금 단계:** 잔액(balance)을 출금액(amount) 만큼 줄인다.

```

출금() {
    1. 검증 단계: 잔액(balance) 확인
    2. 출금 단계: 잔액(balance) 감소
}

```

이 로직에는 하나의 큰 가정이 있다.

스레드 하나의 관점에서 출금() 을 보면 1. 검증 단계에서 확인한 잔액(balance) 1000원은 2. 출금 단계에서 계산을 끝마칠 때 까지 같은 1000원으로 유지되어야 한다. 그래야 검증 단계에서 확인한 금액으로, 출금 단계에서 정확한 잔액을 계산할 수 있다.

그래야 검증 단계에서 확인한 1000원에 800원을 차감해서 200원이라는 잔액을 정확하게 계산할 수 있다.

결국 여기서는 내가 사용하는 값이 중간에 변경되지 않을 것이라는 가정이 있다.

그런데 만약 중간에 다른 스레드가 잔액의 값을 변경한다면, 큰 혼란이 발생한다. 1000원이라 생각한 잔액이 다른 값으로 변경되면 잔액이 전혀 다른 값으로 계산될 수 있다.

공유 자원

잔액(balance)은 여러 스레드가 함께 사용하는 공유 자원이다. 따라서 출금 로직을 수행하는 중간에 다른 스레드에서 이 값을 얼마든지 변경할 수 있다. 참고로 여기서는 출금() 메서드를 호출할 때만 잔액(balance)의 값이 변경된다. 따라서 다른 스레드가 출금 메서드를 호출하면서, 사용중인 출금 값을 중간에 변경해 버릴 수 있다.

한 번에 하나의 스레드만 실행

만약 출금()이라는 메서드를 한 번에 하나의 스레드만 실행할 수 있게 제한한다면 어떻게 될까?

예를 들어 t1, t2 스레드가 함께 출금()을 호출하면 t1 스레드가 먼저 처음부터 끝까지 출금() 메서드를 완료하고, 그 다음에 t2 스레드가 처음부터 끝까지 출금() 메서드를 완료하는 것이다.

이렇게 하면 공유 자원인 balance를 한 번에 하나의 스레드만 변경할 수 있다. 따라서 계산 중간에 다른 스레드가 balance의 값을 변경하는 부분을 걱정하지 않아도 된다. (참고로 여기서는 출금() 메서드를 호출할 때만 잔액(balance)의 값이 변경된다.)

- 더 자세히는 출금을 진행할 때 잔액(balance)을 검증하는 단계부터 잔액의 계산을 완료할 때 까지 잔액의 값은 중간에 변하면 안된다.
- 이 검증과 계산 이 두 단계는 한 번에 하나의 스레드만 실행해야 한다. 그래야 잔액(balance)이 중간에 변하지 않고, 안전하게 계산을 수행할 수 있다.

임계 영역(critical section)

영어로 크리티컬 섹션이라 한다.

- 여러 스레드가 동시에 접근하면 데이터 불일치나 예상치 못한 동작이 발생할 수 있는 위험하고 또 중요한 코드 부분을 뜻한다.
- 여러 스레드가 동시에 접근해서는 안 되는 공유 자원을 접근하거나 수정하는 부분을 의미한다.
 - 예) 공유 변수나 공유 객체를 수정

앞서 우리가 살펴본 출금() 로직이 바로 임계 영역이다.

더 자세히는 출금을 진행할 때 잔액(balance)을 검증하는 단계부터 잔액의 계산을 완료할 때 까지가 임계 영역이다. 여기서 balance는 여러 스레드가 동시에 접근해서는 안되는 공유 자원이다.

이런 임계 영역은 한 번에 하나의 스레드만 접근할 수 있도록 안전하게 보호해야 한다.

그럼 어떻게 한 번에 하나의 스레드만 접근할 수 있도록 임계 영역을 안전하게 보호할 수 있을까?

여러가지 방법이 있지만 자바는 `synchronized` 키워드를 통해 아주 간단하게 임계 영역을 보호할 수 있다.

synchronized 메서드

자바의 `synchronized` 키워드를 사용하면 한 번에 하나의 스레드만 실행할 수 있는 코드 구간을 만들 수 있다.

`BankAccountV1` 을 복사해서 `BankAccountV2` 코드를 만들고 `synchronized` 를 도입해보자.

```
package thread.sync;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class BankAccountV2 implements BankAccount {
    private int balance;

    public BankAccountV2(int initialBalance) {
        this.balance = initialBalance;
    }

    @Override
    public synchronized boolean withdraw(int amount) {
        log("거래 시작: " + getClass().getSimpleName());

        log("[검증 시작] 출금액: " + amount + ", 잔액: " + balance);
        if (balance < amount) {
            log("[검증 실패] 출금액: " + amount + ", 잔액: " + balance);
            return false;
        }
        log("[검증 완료] 출금액: " + amount + ", 잔액: " + balance);
        sleep(1000);
        balance = balance - amount;
        log("[출금 완료] 출금액: " + amount + ", 변경 잔액: " + balance);

        log("거래 종료");
        return true;
    }

    @Override
```

```

    public synchronized int getBalance() {
        return balance;
    }
}

```

- BankAccountV1 와 같은데, withdraw(), getBalance() 코드에 synchronized 키워드가 추가되었다.
- 이제 withdraw(), getBalance() 메서드는 한 번에 하나의 스레드만 실행할 수 있다.

```

public class BankMain {
    public static void main(String[] args) throws InterruptedException {
        //BankAccount account = new BankAccountV1(1000);
        BankAccount account = new BankAccountV2(1000);
        ...
    }
}

```

- BankMain 에서 BankAccountV2 를 실행하도록 코드를 변경하자.

실행 결과

```

14:48:15.144 [      t1] 거래 시작: BankAccountV2
14:48:15.149 [      t1] [검증 시작] 출금액: 800, 잔액: 1000
14:48:15.149 [      t1] [검증 완료] 출금액: 800, 잔액: 1000
14:48:15.633 [    main] t1 state: TIMED_WAITING
14:48:15.633 [    main] t2 state: BLOCKED
14:48:16.155 [      t1] [출금 완료] 출금액: 800, 변경 잔액: 200
14:48:16.155 [      t1] 거래 종료
14:48:16.156 [      t2] 거래 시작: BankAccountV2
14:48:16.156 [      t2] [검증 시작] 출금액: 800, 잔액: 200
14:48:16.156 [      t2] [검증 실패] 출금액: 800, 잔액: 200
14:48:16.160 [    main] 최종 잔액: 200

```

실행 결과를 보면 t1 이 withdraw() 메서드를 시작부터 완료까지 모두 끝내고 나서, 그 다음에 t2 가 withdraw() 메서드를 수행하는 것을 확인할 수 있다. 물론 환경에 따라 t2 가 먼저 실행될 수도 있다. 이 경우에도 t2 가 withdraw() 메서드를 모두 수행한 다음에 t1 이 withdraw() 메서드를 수행한다.

synchronized 분석

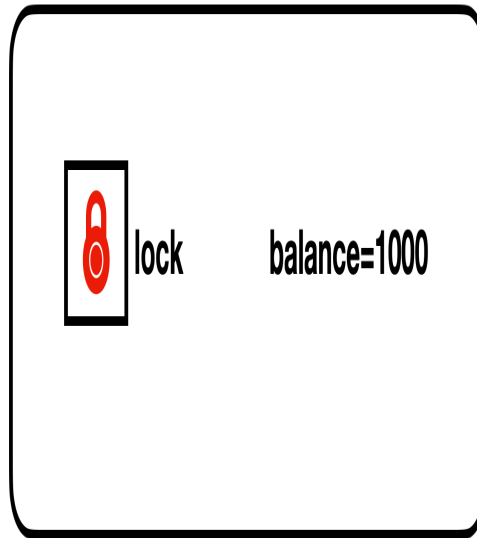
지금부터 자바의 `synchronized`가 어떻게 작동하는지 그림으로 분석해보자.

참고로 실행 결과를 보면 `t2`가 `BLOCKED` 상태인데, 이 상태도 확인해보자.

synchronized t2 t1

```
synchronized withdraw {  
    if (balance < amount) {  
        return false;  
    }  
    sleep(1000);  
    balance = balance - amount;  
}
```

x001



BankAccount

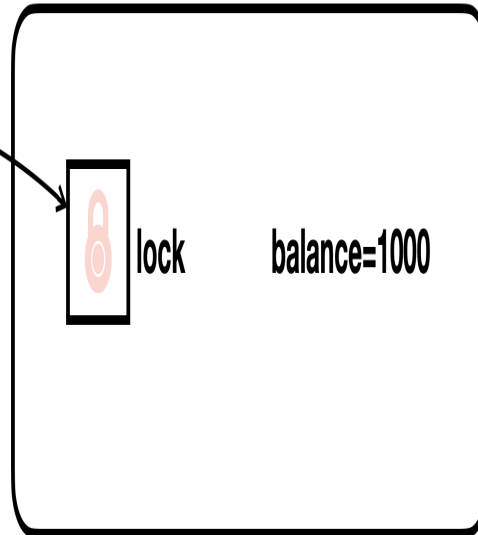
- 모든 객체(인스턴스)는 내부에 자신만의 락(`lock`)을 가지고 있다.
 - 모니터 락(`monitor lock`)이라고도 부른다.
 - 객체 내부에 있고 우리가 확인하기는 어렵다.
- 스레드가 `synchronized` 키워드가 있는 메서드에 진입하려면 반드시 해당 인스턴스의 락이 있어야 한다!
 - 여기서는 `BankAccount(x001)` 인스턴스의 `synchronized withdraw()` 메서드를 호출하므로 이 인스턴스의 락이 필요하다.
- 스레드 `t1`, `t2`는 `withdraw()`를 실행하기 직전이다.

synchronized t2

```
synchronized withdraw {  
  if (balance < amount) {  
    return false;  
  }  
  sleep(1000);  
  balance = balance - amount;  
}
```

t1: lock 획득

x001



BankAccount

- t1 이 먼저 실행된다고 가정하겠다.
- 스레드 t1 이 먼저 synchronized 키워드가 있는 withdraw() 메서드를 호출한다.
- synchronized 메서드를 호출하려면 먼저 해당 인스턴스의 락이 필요하다.
- 락이 있으므로 스레드 t1 은 BankAccount(x001) 인스턴스에 있는 락을 획득한다.

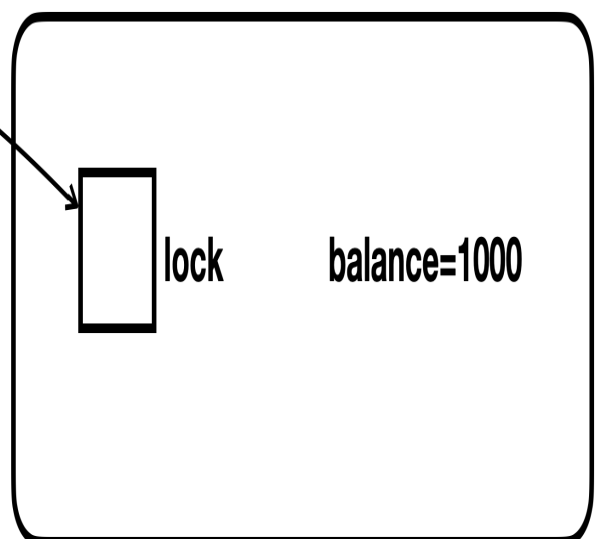
1. t2: lock 획득 시도 -> lock이 없음

2. t2: lock을 획득할 때 까지 BLOCKED 상태로 대기

synchronized

```
synchronized withdraw {  
  if (balance < amount) {  
    return false;  
  }  
  sleep(1000);  
  balance = balance - amount;  
}
```

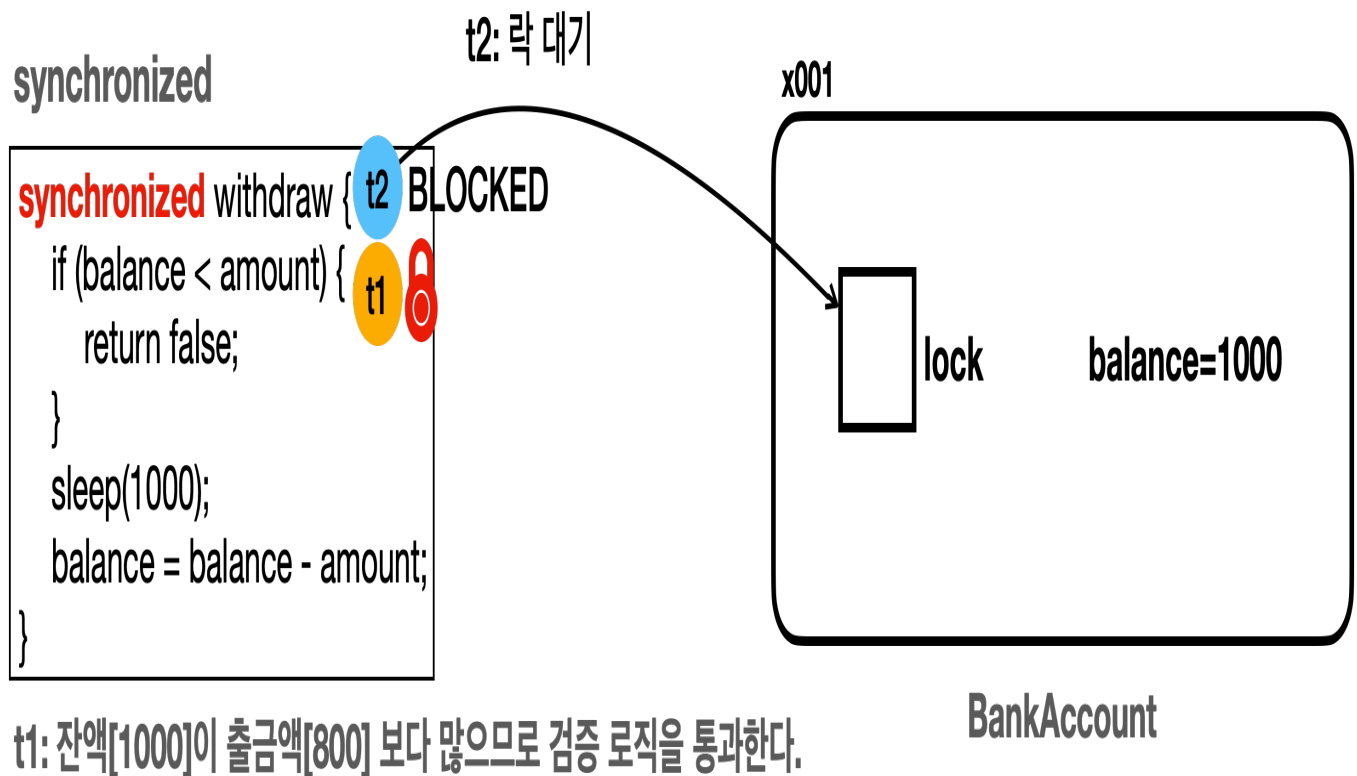
x001



BankAccount

- 스레드 t1 은 해당 인스턴스의 락을 획득했기 때문에 withdraw() 메서드에 진입할 수 있다.
- 스레드 t2 도 withdraw() 메서드 호출을 시도한다. synchronized 메서드를 호출하려면 먼저 해당 인스턴스의 락이 필요하다.
- 스레드 t2 는 BankAccount(x001) 인스턴스에 있는 락 획득을 시도한다. 하지만 락이 없다. 이렇게 락이 없으면 t2 스레드는 락을 획득할 때 까지 BLOCKED 상태로 대기한다.
 - t2 스레드의 상태는 RUNNABLE → BLOCKED 상태로 변하고, 락을 획득할 때 까지 무한정 대기한다.

참고로 BLOCKED 상태가 되면 락을 다시 획득하기 전까지는 계속 대기하고, CPU 실행 스케줄링에 들어가지 않는다.



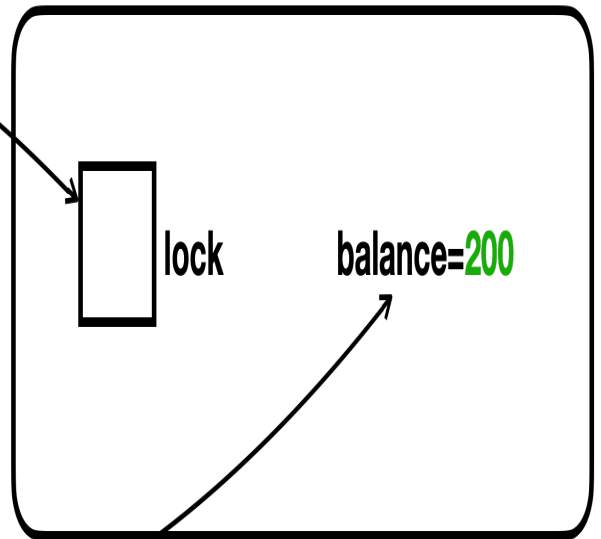
- t1: 출금을 위한 검증 로직을 수행한다. 조건을 만족하므로 검증 로직을 통과한다.
 - 잔액[1000]이 출금액[800] 보다 많으므로 통과한다.

synchronized

t2: 락 대기

```
synchronized withdraw { t2 BLOCKED
  if (balance < amount) {
    return false;
  }
  sleep(1000);
  balance = balance - amount; t1
}
```

x001



t1: balance[200] = balance[1000] - amount[800]

200

BankAccount

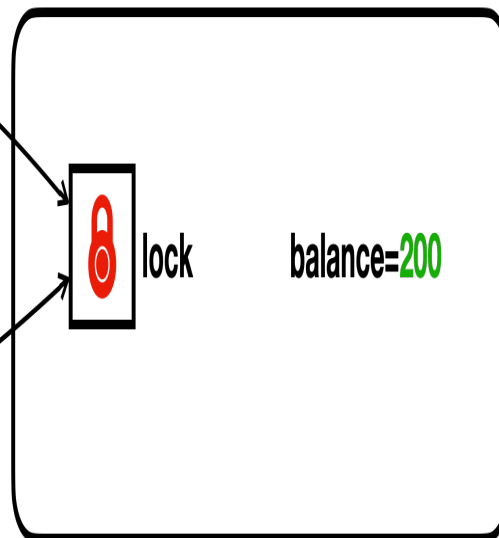
- t1: 잔액 1000원에서 800원을 출금하고 계산 결과인 200원을 잔액(balance)에 반영한다.

synchronized

t2: 락 대기

```
synchronized withdraw { t2 BLOCKED
  if (balance < amount) {
    return false;
  }
  sleep(1000);
  balance = balance - amount;
} t1
```

x001



BankAccount

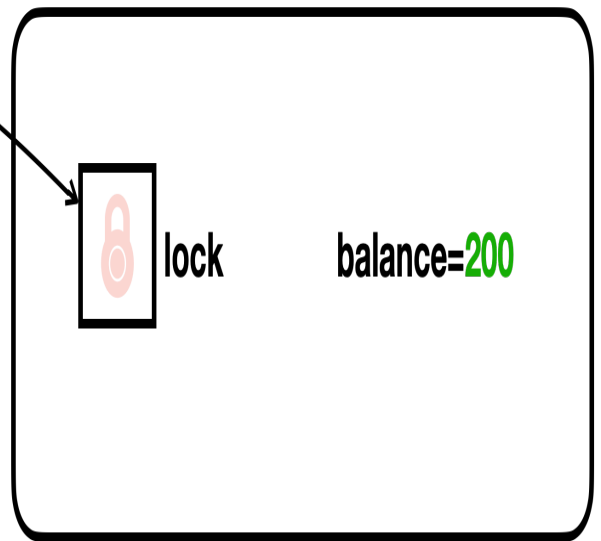
- t1: 메서드 호출이 끝나면 락을 반납한다.

synchronized

t2: 락 획득

x001

```
synchronized withdraw {  
  if (balance < amount) {  
    return false;  
  }  
  sleep(1000);  
  balance = balance - amount;  
} t1
```



BankAccount

t2: 락 획득하면서

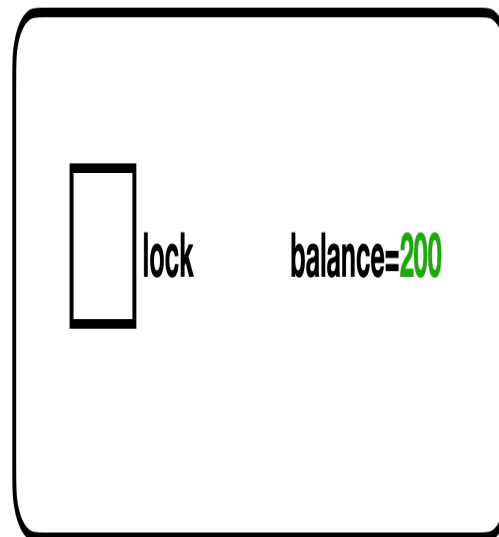
BLOCKED -> RUNNABLE

- t2 : 인스턴스에 락이 반납되면 락 획득을 대기하는 스레드는 자동으로 락을 획득한다.
 - 이때 락을 획득한 스레드는 BLOCKED → RUNNABLE 상태가 되고, 다시 코드를 실행한다.

synchronized

x001

```
synchronized withdraw {  
  if (balance < amount) { t2 }  
    return false;  
  }  
  sleep(1000);  
  balance = balance - amount;  
} t1
```



BankAccount

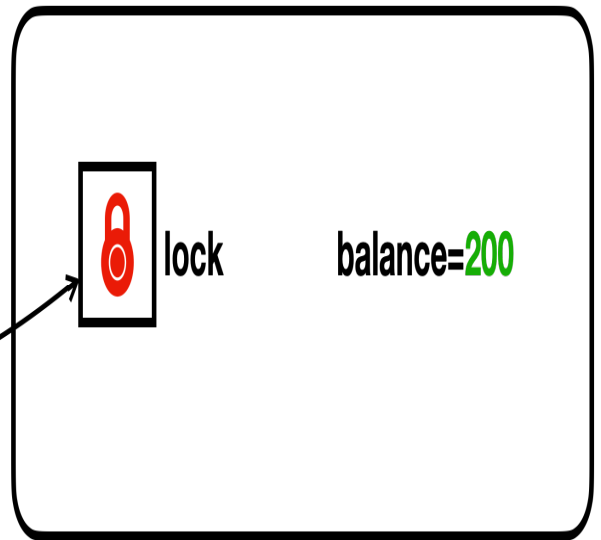
t2: 잔액[200]이 출금액[800]보다 적으므로 검증 로직을 통과하지 못한다.

- 스레드 t2 는 해당 인스턴스의 락을 획득했기 때문에 withdraw() 메서드에 진입할 수 있다.
- t2 : 출금을 위한 검증 로직을 수행한다. 조건을 만족하지 않으므로 false 를 반환한다.
 - 이때 잔액(balance)은 200원이다. 800원을 출금해야 하므로 조건을 만족하지 않는다.

synchronized

```
synchronized withdraw {  
  if (balance < amount) {  
    return false; t2  
  }  
  sleep(1000);  
  balance = balance - amount;  
} t1
```

x001



BankAccount

t2: 락 반납하면서 return

t1: 800원 출금 완료

t2: 잔액 부족으로 출금 실패

원금 1000원, 최종 잔액 200원

- t2: 락을 반납하면서 return 한다.

결과

- t1: 800원 출금 완료
- t2: 잔액 부족으로 출금 실패
- 원금 1000원, 최종 잔액은 200원

t1 은 800원 출금에 성공하지만, t2 는 잔액 부족으로 출금에 실패한다. 그리고 최종 잔액은 1000원에서 200원이 되므로 정확하게 맞다.

이렇게 자바의 `synchronized` 를 사용하면 한 번에 하나의 스레드만 실행하는 안전한 임계 영역 구간을 편리하게 만들 수 있다.

참고: 락을 획득하는 순서는 보장되지 않는다.

만약 `BankAccount(x001)` 인스턴스의 `withdraw()` 를 수 많은 스레드가 동시에 호출한다면, 1개의 스레드만 락을 획득하고 나머지는 모두 `BLOCKED` 상태가 된다. 그리고 이후에 `BankAccount(x001)` 인스턴스에 락을 반납하

면, 해당 인스턴스의 락을 기다리는 수 많은 스레드 중에 하나의 스레드만 락을 획득하고, 락을 획득한 스레드만 **BLOCKED** → **RUNNABLE** 상태가 된다.

이때 어떤 순서로 락을 획득하는지는 자바 표준에 정의되어 있지 않다. 따라서 순서를 보장하지 않고, 환경에 따라서 순서가 달라질 수 있다.

참고: `volatile`를 사용하지 않아도 `synchronized` 안에서 접근하는 변수의 메모리 가시성 문제는 해결된다. (이전에 학습한 자바 메모리 모델 참고)

synchronized 코드 블럭

`synchronized`의 가장 큰 장점이자 단점은 한 번에 하나의 스레드만 실행할 수 있다는 점이다. 여러 스레드가 동시에 실행하지 못하기 때문에, 전체로 보면 성능이 떨어질 수 있다. 따라서 `synchronized`를 통해 여러 스레드를 동시에 실행할 수 없는 코드 구간은 꼭! 필요한 곳으로 한정해서 설정해야 한다.

이전에 작성한 다음 코드를 보자.

```
public synchronized boolean withdraw(int amount) {
    log("거래 시작: " + getClass().getSimpleName());

    log("[검증 시작] 출금액: " + amount + ", 잔액: " + balance);
    if (balance < amount) {
        log("[검증 실패] 출금액: " + amount + ", 잔액: " + balance);
        return false;
    }
    log("[검증 완료] 출금액: " + amount + ", 잔액: " + balance);
    sleep(1000);
    balance = balance - amount;
    log("[출금 완료] 출금액: " + amount + ", 변경 잔액: " + balance);

    log("거래 종료");
    return true;
}
```

처음에 로그를 출력하는 "거래 시작", 그리고 마지막에 로그를 출력하는 "거래 종료" 부분은 공유 자원을 전혀 사용하지 않는다. 이런 부분은 동시에 실행해도 아무 문제가 발생하지 않는다.

따라서 진짜 임계 영역은 다음과 같다.

```

public synchronized boolean withdraw(int amount) {
    log("거래 시작: " + getClass().getSimpleName());

    // ==임계 영역 시작==
    log("[검증 시작] 출금액: " + amount + ", 잔액: " + balance);
    if (balance < amount) {
        log("[검증 실패] 출금액: " + amount + ", 잔액: " + balance);
        return false;
    }
    log("[검증 완료] 출금액: " + amount + ", 잔액: " + balance);
    sleep(1000);
    balance = balance - amount;
    log("[출금 완료] 출금액: " + amount + ", 변경 잔액: " + balance);
    // ==임계 영역 종료==

    log("거래 종료");
    return true;
}

```

그런데 메서드 앞에 적용한 `synchronized`의 적용 범위는 메서드 전체이다. 따라서 여러 스레드가 함께 실행해도 문제가 없는 "거래 시작", "거래 종료"를 출력하는 코드도 한 번에 하나의 스레드만 실행할 수 있다.

자바는 이런 문제를 해결하기 위해 `synchronized`를 메서드 단위가 아니라, 특정 코드 블록에 최적화 해서 적용할 수 있는 기능을 제공한다.

`BankAccountV2`의 코드를 복사해서 `BankAccountV3`를 만들자.

```

package thread.sync;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class BankAccountV3 implements BankAccount {
    private int balance;

    public BankAccountV3(int initialBalance) {
        this.balance = initialBalance;
    }

    @Override

```

```

public boolean withdraw(int amount) {
    log("거래 시작: " + getClass().getSimpleName());

    synchronized (this) {
        log("[검증 시작] 출금액: " + amount + ", 잔액: " + balance);
        if (balance < amount) {
            log("[검증 실패] 출금액: " + amount + ", 잔액: " + balance);
            return false;
        }
        log("[검증 완료] 출금액: " + amount + ", 잔액: " + balance);
        sleep(1000);
        balance = balance - amount;
        log("[출금 완료] 출금액: " + amount + ", 변경 잔액: " + balance);
    }

    log("거래 종료");
    return true;
}

@Override
public synchronized int getBalance() {
    return balance;
}
}

```

- `withdraw()` 메서드 앞에 사용하던 `synchronized`를 제거한다.
- `synchronized (this) {}`: 안전한 임계 영역을 코드 블록으로 지정한다.
- 이렇게 하면 꼭 필요한 코드만 안전한 임계 영역으로 만들 수 있다.
- `synchronized (this)`: 여기서 괄호 () 안에 들어가는 값은 락을 획득할 인스턴스의 참조이다.
 - 여기서는 `BankAccountV3(x001)`의 인스턴스의 락을 사용하므로 이 인스턴스의 참조인 `this`를 넣어 주면 된다.
 - 이전에 메서드에 `synchronized`를 사용할 때와 같은 인스턴스에서 락을 획득한다.
- `getBalance()`의 경우 `return balance` 코드 한 줄이므로 `synchronized`를 메서드에 설정하나 코드 블록으로 설정하나 둘다 같다.

```

public class BankMain {
    public static void main(String[] args) throws InterruptedException {
        //BankAccount account = new BankAccountV2(1000);
        BankAccount account = new BankAccountV3(1000);
        ...
    }
}

```

```
}
```

- BankMain에서 BankAccountV3를 실행하도록 코드를 변경하자.

실행 결과

```
14:20:51.851 [      t1] 거래 시작: BankAccountV3
14:20:51.851 [      t2] 거래 시작: BankAccountV3
14:20:51.858 [      t1] [검증 시작] 출금액: 800, 잔액: 1000
14:20:51.858 [      t1] [검증 완료] 출금액: 800, 잔액: 1000
14:20:52.337 [    main] t1 state: TIMED_WAITING
14:20:52.337 [    main] t2 state: BLOCKED
14:20:52.861 [      t1] [출금 완료] 출금액: 800, 변경 잔액: 200
14:20:52.862 [      t1] 거래 종료
14:20:52.862 [      t2] [검증 시작] 출금액: 800, 잔액: 200
14:20:52.864 [      t2] [검증 실패] 출금액: 800, 잔액: 200
14:20:52.869 [    main] 최종 잔액: 200
```

synchronized 블럭 기능을 사용한 덕분에 딱 필요한 부분에 임계 영역을 지정할 수 있었다. 덕분에 아주 약간이지만 여러 스레드가 동시에 수행되는 부분을 더 늘려서, 전체적으로 성능을 더 향상할 수 있었다. 지금의 예는 단순히 로그 몇 줄 출력하는 정도이지만, 만약 작업이 오래 수행된다면 큰 성능 차이가 발생할 것이다.

여기서는 처음 거래 시작 부분이 t1, t2 동시에 실행된 것을 확인할 수 있다.

```
14:20:51.851 [      t1] 거래 시작: BankAccountV3
14:20:51.851 [      t2] 거래 시작: BankAccountV3
```

여기서 이야기하고 싶은 핵심은 하나의 스레드만 실행할 수 있는 안전한 임계 영역은 가능한 최소한의 범위에 적용해야 한다는 점이다. 그래야 동시에 여러 스레드가 실행할 수 있는 부분을 늘려서, 전체적인 처리 성능을 더 높일 수 있다.

synchronized 동기화 정리

자바에서 동기화(synchronization)는 여러 스레드가 동시에 접근할 수 있는 자원(예: 객체, 메서드)에 대해 일관성 있고 안전한 접근을 보장하기 위한 메커니즘이다. 동기화는 주로 멀티스레드 환경에서 발생할 수 있는 문제, 예를 들어 데이터 손상이나 예기치 않은 결과를 방지하기 위해 사용된다.

메서드 동기화: 메서드를 synchronized로 선언해서, 메서드에 접근하는 스레드가 하나뿐이도록 보장한다.

```
public synchronized void synchronizedMethod() {
    // 코드
}
```

블록 동기화: 코드 블록을 `synchronized` 로 감싸서, 동기화를 구현할 수 있다.

```
public void method() {
    synchronized(this) {
        // 동기화된 코드
    }
}
```

이런 동기화를 사용하면 다음 문제들을 해결할 수 있다.

- **경합 조건(Race condition):** 두 개 이상의 스레드가 경쟁적으로 동일한 자원을 수정할 때 발생하는 문제.
- **데이터 일관성:** 여러 스레드가 동시에 읽고 쓰는 데이터의 일관성을 유지.

동기화는 멀티스레드 환경에서 필수적인 기능이지만, 과도하게 사용할 경우 성능 저하를 초래할 수 있으므로 꼭 필요한 곳에 적절히 사용해야 한다.

문제와 풀이

경고: 반드시 풀어보세요!!!!

문제1 - 공유 자원

다음 코드의 결과는 20000이 되어야 한다.

이 코드의 문제점을 찾아서 해결해라.

이 코드에서 다른 부분은 변경하면 안되고, `Counter` 클래스 내부만 수정해야 한다.

```
package thread.sync.test;

public class SyncTest1BadMain {
```

```

public static void main(String[] args) throws InterruptedException {
    Counter counter = new Counter();

    Runnable task = new Runnable() {
        @Override
        public void run() {
            for (int i = 0; i < 10000; i++) {
                counter.increment();
            }
        }
    };

    Thread thread1 = new Thread(task);
    Thread thread2 = new Thread(task);

    thread1.start();
    thread2.start();
    thread1.join();
    thread2.join();
    System.out.println("결과: " + counter.getCount());
}

static class Counter {
    private int count = 0;

    public void increment() {
        count = count + 1;
    }

    public int getCount() {
        return count;
    }
}
}

```

기대하는 실행 결과

결과: 20000

- 이 예시의 실행 결과는 20000이 아니라 다른 값이 나올 수 있다. 반드시 20000이 출력되어야 한다.

```
public class SyncTest1Main {  
    // 여기에 코드 작성  
}
```

정답

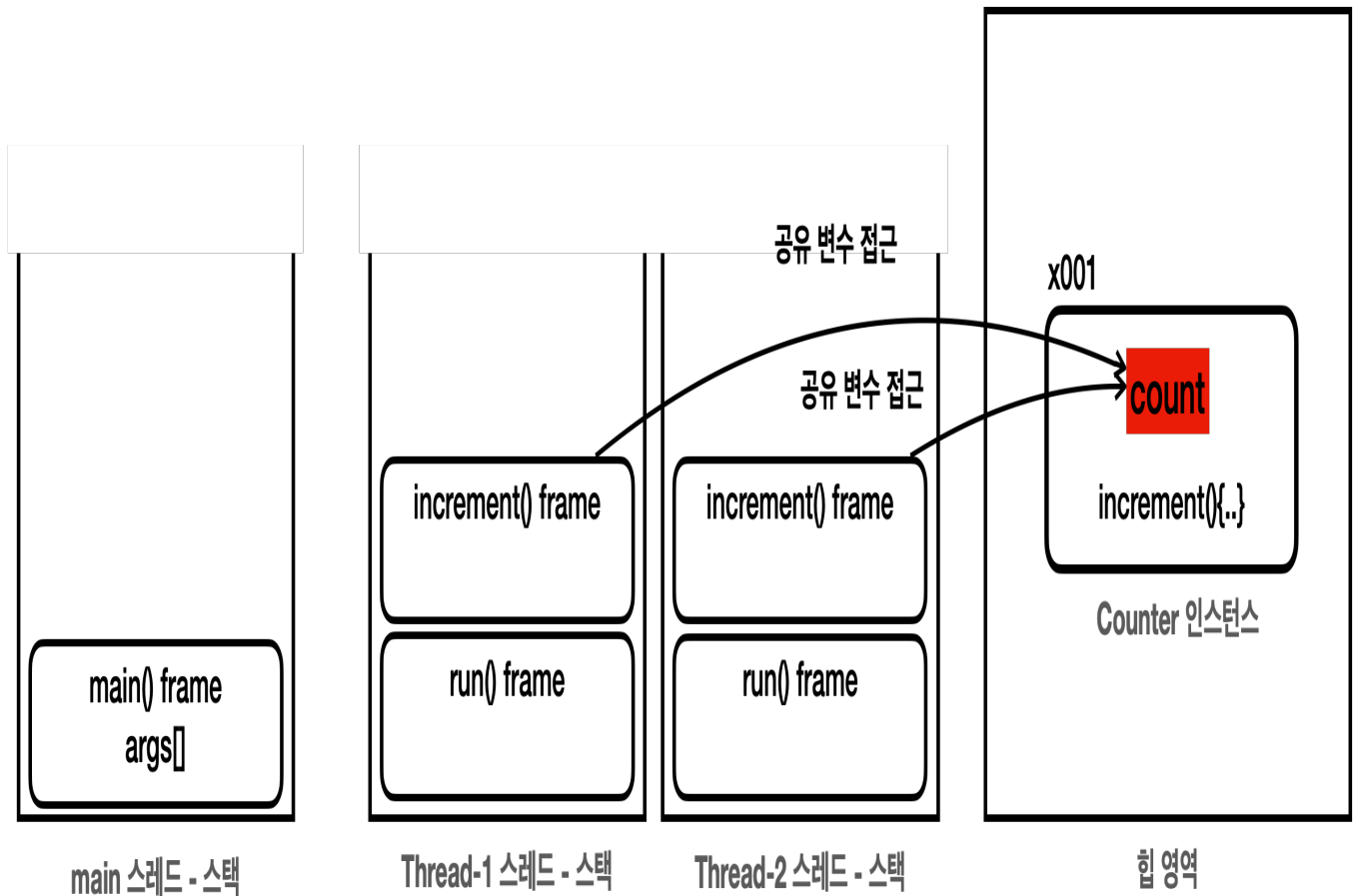
```
package thread.sync.test;  
  
public class SyncTest1Main {  
  
    public static void main(String[] args) throws InterruptedException {  
        Counter counter = new Counter();  
  
        Runnable task = new Runnable() {  
            @Override  
            public void run() {  
                for (int i = 0; i < 10000; i++) {  
                    counter.increment();  
                }  
            }  
        };  
  
        Thread thread1 = new Thread(task);  
        Thread thread2 = new Thread(task);  
  
        thread1.start();  
        thread2.start();  
        thread1.join();  
        thread2.join();  
        System.out.println("결과: " + counter.getCount());  
    }  
  
    static class Counter {  
        private int count = 0;  
  
        public synchronized void increment() {  
            count = count + 1;  
        }  
  
        public synchronized int getCount() {
```

```

    return count;
}
}
}

```

`count` 는 여러 스레드가 함께 사용하는 공유 자원이다.



여기서 공유 자원을 사용하는 `increment()` 메서드는 다음과 같이 3단계로 나누어져 있다.

```
count = count + 1
```

1. `count` 의 값을 읽는다.
2. 읽은 `count` 의 값에 1을 더한다.
3. 더한 결과를 `count` 에 다시 저장한다.

단일 스레드가 공유 자원에 접근하는 상황

`count` 값이 0이라고 가정하겠다.

1. `count` 의 값을 읽는다. `count` 값은 0이다.
2. 읽은 `count` 의 값에 1을 더한다. $0 + 1 = 1$ 이다.
3. 더한 결과를 `count` 에 다시 저장한다. `count` 값은 1이다.

이처럼 단일 스레드가 공유 자원에 접근하는 경우는 아무런 문제가 없다.

여러 스레드가 공유 자원에 함께 접근하는 상황

```
count = count + 1
```

count 값이 0이라고 가정하겠다. 2개의 스레드가 동시에 increment() 메서드를 호출한다.

- 스레드1: count 의 값을 읽는다. count 값은 0이다.
- 스레드2: count 의 값을 읽는다. count 값은 0이다.
- 스레드1: 읽은 count 의 값에 1을 더한다. $0 + 1 = 1$ 이다.
- 스레드2: 읽은 count 의 값에 1을 더한다. $0 + 1 = 1$ 이다.
- 스레드1: 더한 결과를 count 에 다시 저장한다. count 값은 1이다.
- 스레드2: 더한 결과를 count 에 다시 저장한다. count 값은 1이다.

스레드 2개가 increment() 를 호출하기 때문에 기대하는 count 의 결과는 2가 되어야 한다.

하지만 둘이 동시에 실행되기 때문에, 처음에 둘다 count 의 값을 0으로 읽었다.

여기서 잘 보면 count 의 값을 읽어서 계산하는 부분과 그 결과를 count 에 다시 넣는 부분으로 나누어져 있다.

따라서 여러 스레드가 동시에 실행되면 지금과 같은 문제가 발생할 수 있다.

따라서 synchronized 키워드를 사용해서 한 번에 하나의 스레드만 실행할 수 있도록, 안전한 임계 영역을 만들어야 한다.

문제2 - 지역 변수의 공유

다음 코드에서 MyTask 의 run() 메서드는 두 스레드에서 동시에 실행한다.

다음 코드의 실행 결과를 예측해보자.

그리고 localValue 지역 변수에 동시성 문제가 발생하는지 하지 않는지 생각해보자.

```
package thread.sync.test;

import static util.MyLogger.log;

public class SyncTest2Main {
    public static void main(String[] args) throws InterruptedException {
```

```

MyCounter myCounter = new MyCounter();

Runnable task = new Runnable() {
    @Override
    public void run() {
        myCounter.count();
    }
};

Thread thread1 = new Thread(task, "Thread-1");
Thread thread2 = new Thread(task, "Thread-2");

thread1.start();
thread2.start();
}

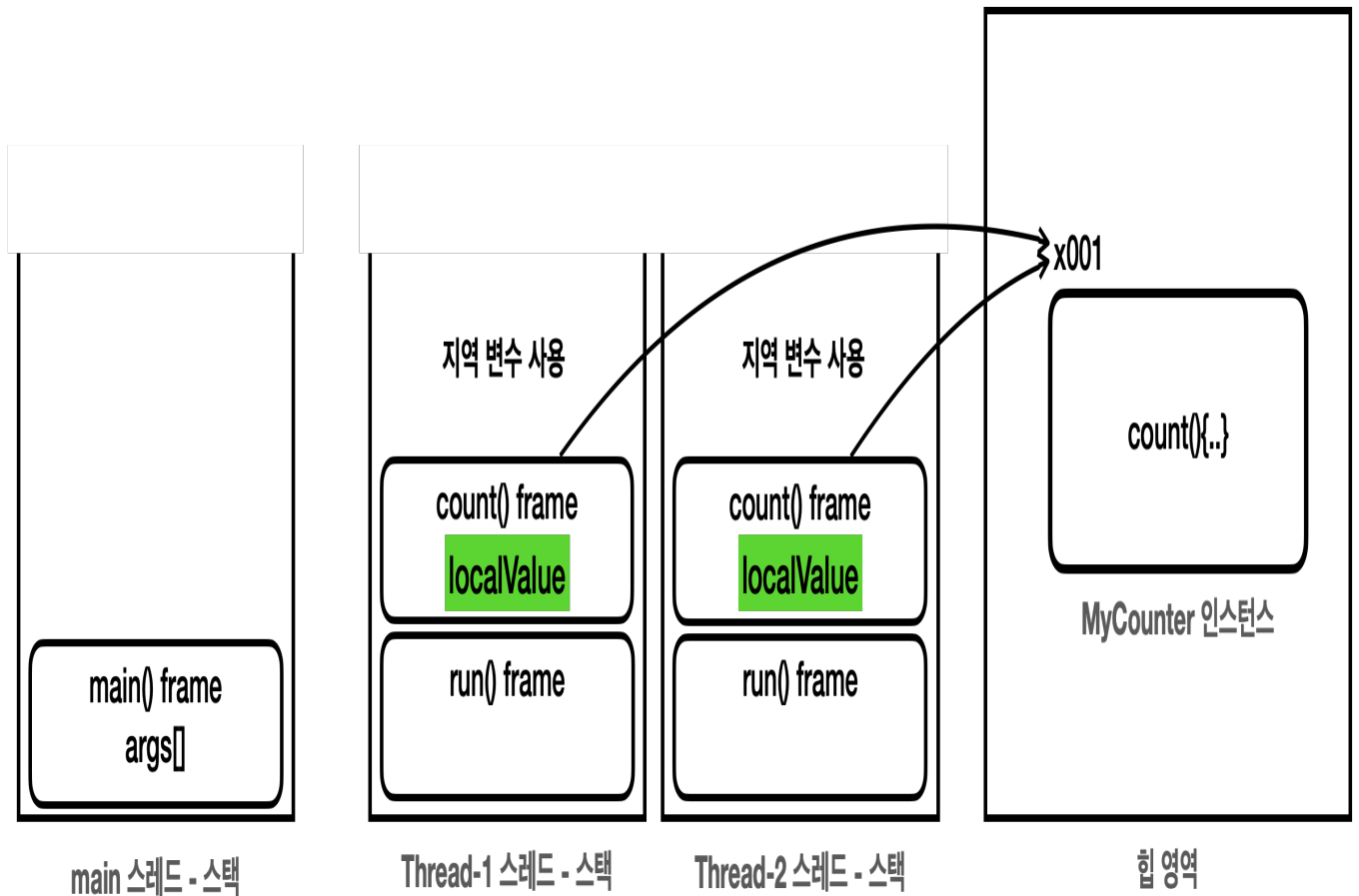
static class MyCounter {

    public void count() {
        int localValue = 0;
        for (int i = 0; i < 1000; i++) {
            localValue = localValue + 1;
        }
        log("결과: " + localValue);
    }
}

}

```

정답



- `localValue` 는 지역 변수이다.
- 스택 영역은 각각의 스레드가 가지는 별도의 메모리 공간이다. 이 메모리 공간은 다른 스레드와 공유하지 않는다.
- 지역 변수는 스레드의 개별 저장 공간인 스택 영역에 생성된다.
- 따라서 **지역 변수는 절대로! 다른 스레드와 공유되지 않는다!**
- 이런 이유로 지역 변수는 동기화에 대한 걱정을 하지 않아도 된다.
 - 여기에 `synchronized` 를 사용하면 아무 이득도 얻을 수 없다. 성능만 느려진다!
- 지역 변수를 제외한, 인스턴스의 멤버 변수(필드), 클래스 변수 등은 공유될 수 있다.

문제3 - final 필드

다음에서 `value` 필드(멤버 변수)는 공유되는 값이다. 멀티스레드 상황에서 문제가 될 수 있을까?

```
class Immutable {
    private final int value;

    public Immutable(int value) {
        this.value = value;
    }

    public int getValue() {
```

```
        return value;
    }

}
```

정답

여러 스레드가 공유 자원에 접근하는 것 자체는 사실 문제가 되지 않는다. 진짜 문제는 공유 자원을 사용하는 중간에 다른 스레드가 공유 자원의 값을 변경해버리기 때문에 발생한다. 결국 변경이 문제가 되는 것이다.

여러 스레드가 접근 가능한 공유 자원이라도 그 값을 아무도 변경할 수 없다면 문제 되지 않는다. 이 경우 모든 스레드가 항상 같은 값을 읽기 때문이다.

필드에 `final` 이 붙으면 어떤 스레드도 값을 변경할 수 없다. 따라서 멀티스레드 상황에 문제 없는 안전한 공유 자원이 된다.

정리

자바는 처음부터 멀티스레드를 고려하고 나온 언어이다. 그래서 자바 1.0 부터 `synchronized` 같은 동기화 방법을 프로그래밍 언어의 문법에 포함해서 제공한다.

`synchronized` 장점

- 프로그래밍 언어에 문법으로 제공
- 아주 편리한 사용
- **자동 잠금 해제**: `synchronized` 메서드나 블록이 완료되면 자동으로 락을 대기중인 다른 스레드의 잠금이 해제된다. 개발자가 직접 특정 스레드를 깨우도록 관리해야 한다면, 매우 어렵고 번거로울 것이다.

`synchronized` 는 매우 편리하지만, 제공하는 기능이 너무 단순하다는 단점이 있다. 시간이 점점 지나면서 멀티스레드가 더 중요해지고 점점 더 복잡한 동시성 개발 방법들이 필요해졌다.

`synchronized` 단점

- **무한 대기**: `BLOCKED` 상태의 스레드는 락이 풀릴 때 까지 무한 대기한다.
 - 특정 시간까지만 대기하는 타임아웃X
 - 중간에 인터럽트X
- **공정성**: 락이 돌아왔을 때 `BLOCKED` 상태의 여러 스레드 중에 어떤 스레드가 락을 획득할 지 알 수 없다. 최악의

경우 특정 스레드가 너무 오랜기간 락을 획득하지 못할 수 있다.

`synchronized`의 가장 치명적인 단점은 락을 얻기 위해 `BLOCKED` 상태가 되면 락을 얻을 때까지 무한 대기한다는 점이다. 비유를 하자면 맛집에 한 번 줄을 서면 10시간이든 100시간이든 밥을 먹을 때까지 강제적으로 계속 기다려야 한다는 점이다.

예를 들어 웹 애플리케이션의 경우 고객이 어떤 요청을 했는데, 화면에 계속 요청 중만 뜨고, 응답을 못 받는 것이다. 차라리 너무 오랜 시간이 지나면, 시스템에 사용자가 너무 많아서 다음에 다시 시도해달라고 하는 식의 응답을 주는 것이 더 나은 선택일 것이다.

결국 더 유연하고, 더 세밀한 제어가 가능한 방법들이 필요하게 되었다. 이런 문제를 해결하기 위해 자바 1.5부터 `java.util.concurrent`라는 동시성 문제 해결을 위한 패키지가 추가된다.

참고로 단순하고 편리하게 사용하기에는 `synchronized`가 좋으므로, 목적에 부합한다면 `synchronized`를 사용하면 된다.